

Plan Maestro SaaS DGII

Facturación Electrónica e-CF para República Dominicana

Versión: 1.0 **Fecha:** 2026-05-21 **Autor:** Equipo SaaS DGII (basado en lecciones de DermaLand)

Este documento consolida cinco entregables independientes:

1. Plan Maestro estratégico y operativo.
2. PRD (Product Requirements Document) detallado por pantalla y por estado.
3. Catálogo de 25 agentes especializados + skills + prompts reusables.
4. Roadmap de 20 fases con criterios de salida y bloqueos.
5. Checklist práctico de implementación con responsables y evidencias.

El público objetivo incluye equipo técnico, dueño del SaaS, contador/asesor fiscal, soporte y auditor.

Garantía de seguridad: este documento NO contiene secretos, contraseñas, tokens, RNCs reales, contenido de certificados ni datos sensibles. Las URLs DGII (ecf.dgii.gov.do/*) son documentación pública oficial.

Índice

Plan Maestro SaaS DGII

Facturación Electrónica e-CF para República Dominicana

Índice

Capítulo 1 — Plan Maestro

Plan Maestro SaaS DGII — Facturación Electrónica e-CF para República Dominicana

1. Portada

2. Resumen ejecutivo

3. Visión del sistema

4. Objetivo del módulo

5. Alcance funcional

6. Alcance técnico

7. Alcance fiscal

8. Qué queda fuera del alcance

9. Principios del sistema

10. Arquitectura SaaS multi-tenant recomendada

11. Modelo de datos mínimo

12. Separación por business_id

13. RLS por tenant

14. Auditoría por tenant

15. Seguridad de certificados

16. Seguridad de secretos

17. Seguridad de logs

18. Flujo completo del cliente SaaS

19. Flujo completo técnico

20. Dependencias externas

21. Reglas de seguridad obligatorias

22. Auditoría

23. Manejo de errores DGII

24. Riesgos críticos

25. Decisiones técnicas recomendadas

26. Checklist antes de producción

27. Estrategia de soporte

28. Estrategia de onboarding de clientes

29. Estrategia de monitoreo

30. Estrategia de recuperación ante errores

31. Estrategia para múltiples clientes SaaS

32. Cómo adaptar el módulo a otro sistema

33. Lecciones aprendidas de DermaLand

34. Glosario

35. Anexos

Capítulo 2 — PRD / Product Requirements

PRD — SaaS de Facturación Electrónica DGII para República Dominicana

1. Nombre del producto

2. Problema

3. Oportunidad

4. Usuarios objetivo

5. Jobs to be done

6. Objetivos de producto

7. Objetivos técnicos

8. Objetivos fiscales

9. Métricas de éxito

10. Casos de uso

11. User stories

12. Requisitos funcionales

13. Requisitos no funcionales

14. Requisitos legales / fiscales

15. Requisitos de seguridad

16. Requisitos de auditoría

17. Requisitos multi-tenant

18. Flujos UX

- 19. Estados del proceso
- 20. Requisitos por pantalla
- 21. Criterios de aceptación por pantalla
- 22. Criterios de aceptación por fase
- 23. Casos de error
- 24. Mensajes para usuario final
- 25. Mensajes para contador
- 26. Mensajes para soporte
- 27. Requisitos de documentación
- 28. Riesgos y mitigaciones
- 29. Dependencias
- 30. Fuera de alcance
- 31. Criterios para go-live

Capítulo 3 — Agentes y Skills

Agentes y Skills — SaaS DGII e-CF para RD

- 1. Introducción
- 2. Los 25 agentes
- 3. Catálogo de Skills
- 4. Prompts reutilizables transversales
- 5. Matriz fase → agentes
- 6. Cómo replicarlo

Capítulo 4 — Roadmap por Fases

Roadmap por Fases — SaaS DGII e-CF para RD

Tabla resumen de las 20 fases

Detalle por fase

- 3. Reglas de transición entre fases
- 4. Adaptación a otro SaaS

Capítulo 5 — Checklist de Implementación

Checklist de Implementación — SaaS DGII e-CF para RD

Introducción

Checklist 1 — Inicial del proyecto

Checklist 2 — Arquitectura

Checklist 3 — Supabase

Checklist 4 — RLS

Checklist 5 — Vercel

Checklist 6 — Variables de entorno

Checklist 7 — Multi-tenant

Checklist 8 — Certificados

Checklist 9 — Cifrado

Checklist 10 — DGII

Checklist 11 — RNC / Contribuyente

Checklist 12 — Usuario Administrador e-CF

Checklist 13 — Secuencias e-NCF

Checklist 14 — XML / XSD

Checklist 15 — Firma digital

Checklist 16 — PDF / representación impresa

Checklist 17 — QR / código de seguridad

Checklist 18 — testecf

Checklist 19 — TrackId / status

Checklist 20 — Certificación

Checklist 21 — Producción fiscal

Checklist 22 — Seguridad

Checklist 23 — Auditoría

Checklist 24 — QA automatizado

Checklist 25 — QA manual

Checklist 26 — Soporte cliente

Checklist 27 — Contador

Checklist 28 — Go-live

Checklist 29 — Post-producción

Checklist 30 — Incidentes

Capítulo 1 — Plan Maestro

Plan Maestro SaaS DGII — Facturación Electrónica e-CF para República Dominicana

Versión: 1.0 **Fecha:** 2026-05-21 **Autor:** Equipo SaaS DGII (basado en las lecciones aprendidas implementando DermaLand) **Audiencia:** Equipo técnico, dueño del SaaS, contador / asesor fiscal, soporte y auditor.

1. Portada

Este documento es el primero de un paquete de cinco entregables que conforman el Plan Maestro para construir, certificar y operar un módulo SaaS de facturación electrónica DGII (e-CF) para República Dominicana.

El paquete completo está pensado para ser reutilizable: un equipo nuevo, en un SaaS distinto, debería poder usar estos cinco documentos como guía para implementar el módulo desde cero hasta producción fiscal, respetando los gates de seguridad y compliance que ya validamos en DermaLand.

Documentos del paquete:

1. `plan-maestro-saas-dgii.md` (este documento) — visión estratégica y operativa.
2. `prd-saas-facturacion-electronica-dgii.md` — Product Requirements detallado.
3. `agentes-y-skills-saas-dgii.md` — catálogo de 25 agentes + skills + prompts.
4. `roadmap-fases-saas-dgii.md` — 20 fases con criterios de salida.
5. `checklist-implementacion-saas-dgii.md` — checklists prácticos con responsables.

Garantía de seguridad de este documento: NO contiene secretos, contraseñas, tokens, RNCs reales, contenido de certificados ni datos sensibles. Las URLs `ecf.dgii.gov.do/*` son documentación pública oficial DGII.

2. Resumen ejecutivo

La facturación electrónica e-CF es obligatoria en República Dominicana para un volumen creciente de contribuyentes según la implementación progresiva definida por la DGII (Norma 06-2018 y sus actualizaciones). Cada contribuyente que llegue a su fecha de obligatoriedad debe ser capaz de emitir comprobantes fiscales electrónicos firmados con certificado digital, validados contra el XSD oficial, enviados al ambiente correspondiente (`testecf` / `certecf` / `ecf`) y consultados por TrackId, conservando evidencias por al menos diez años.

Implementar esto desde cero en un SaaS multi-tenant tiene tres dimensiones críticas que no se pueden separar: **técnica** (XML, XMLDSig, criptografía PKCS#12, integración HTTP con DGII), **fiscal** (postulación,

designación de Usuario Administrador e-CF, rangos e-NCF, conservación legal) y **operativa** (onboarding del cliente final no-técnico, soporte, monitoreo, respuesta a incidentes DGII).

Este Plan Maestro consolida una arquitectura, un modelo de datos, un flujo de fases y un conjunto de killswitches que aíslan cada riesgo: el cliente puede subir un certificado real sin que el sistema haga nunca una llamada a DGII; puede preparar un envío completo en modo dry-run sin consumir secuencias; puede certificarse en testecf antes de tocar producción fiscal. El sistema acompaña al cliente paso a paso y no se mueve solo aunque técnicamente pudiera.

3. Visión del sistema

El estado final deseado es un módulo SaaS multi-tenant donde:

- Cualquier contribuyente RD se registra, configura sus datos fiscales y carga su certificado digital en menos de 30 minutos sin asistencia técnica.
 - Cada cliente opera bajo su propio `business_id`, con aislamiento garantizado por Row-Level Security (RLS) en Supabase: cero data leak cross-tenant.
 - El proceso de habilitación es un wizard de 10 pasos con explicaciones claras, evidencia por ítem, y un panel de estado global con 8 estados nombrados que cualquier usuario no-técnico entiende.
 - El envío real a DGII está protegido por killswitches en capas: env flag + chequeo de ambiente + postulación confirmada + rango autorizado + confirmación manual + permiso de rol. Ninguno se asume; todos se verifican.
 - Errores DGII se muestran tal como vienen, sin ocultar — el cliente y su contador ven exactamente qué dijo DGII y qué hacer al respecto.
 - Producción fiscal nunca se activa por accidente: requiere transición explícita testecf → certecf → ecf con autorización per-cliente.
-

4. Objetivo del módulo

Habilitar a cada contribuyente cliente del SaaS para emitir, firmar, enviar y conservar comprobantes fiscales electrónicos DGII de forma segura, auditable y reutilizable entre proyectos.

- Configurar datos fiscales sin tocar código.
 - Subir y custodiar certificado digital cifrado server-side.
 - Generar XML e-CF XSD-compliant para los tipos 31/32/33/34 (y opcionalmente 41-47).
 - Firmar con XMLDSig enveloped contra el cert del Usuario Administrador e-CF.
 - Enviar a testecf / certecf / ecf con auth flow Semilla → ValidarSemilla → bearer token.
-

5. Alcance funcional

Lo que el cliente final puede hacer dentro del módulo:

- Configurar RNC, razón social, dirección fiscal, provincia/municipio, contacto, ambiente activo.
 - Subir certificado `.p12` o `.pfx` cifrado server-side; validarlo localmente con prueba de firma.
 - Cargar rangos e-NCF autorizados por DGII para cada tipo de comprobante.
 - Completar checklist de habilitación de 10 pasos con evidencia por ítem.
 - Ejecutar pruebas de pre-certificación local (XSD válido + firma válida + QR demo).
 - Preparar pre-flight de envío a testecf en modo dry-run (no toca DGII).
 - Enviar comprobantes reales a testecf cuando la postulación esté aprobada.
 - Consultar TrackId y estado de cada envío.
 - Generar PDFs de representación impresa con QR y código de seguridad.
 - Recibir e-CF de terceros (sender) y aprobarlos comercialmente.
 - Crear notas de crédito (e-CF 34) sobre comprobantes propios.
 - Acceder a reportes fiscales por período.
 - Asignar permisos DGII por rol (admin / cajero / contador / soporte).
-

6. Alcance técnico

- **Stack:** Next.js 15 App Router (Server Components + Server Actions + Route Handlers), React 19, TypeScript estricto, Tailwind, Supabase (Postgres + Auth + Storage + RLS), Vercel hosting (Edge + Node Functions), Vitest para tests, Playwright para E2E.
 - **Librerías DGII clave:** `node-forge` (PKCS#12 parsing, RSA sign), `xml-crypto` (XMLDSig enveloped), `xmllint-wasm` (validación XSD oficial), `xmlbuilder2` (construcción de XML), `qrcode` (QR de representación impresa).
 - **Persistencia:** Postgres con RLS por `business_id`. Storage privado para blobs de certificados cifrados (AES-256-GCM).
 - **Integraciones externas:** DGII (HTTP/HTTPS a `ecf.dgii.gov.do/*`), entidad certificadora INDOTEL/DGII (offline; el cert se compra y se sube).
 - **Build & deploy:** monorepo con `apps/web` como Root Directory; Vercel autodeploy desde rama; `outputFileTracingIncludes` para bundlear los XSDs.
 - **Tests:** ≥80% cobertura en builder/signer/validator; tests cross-tenant que rechazan inserts ajenos; tests con `vi.spyOn(global, 'fetch')` que prohíben llamadas accidentales a DGII en suites locales.
-

7. Alcance fiscal

- **Norma DGII 06-2018** y sus actualizaciones (consultar siempre la última vigente).
- **Tipos de e-CF cubiertos:** 31 (Crédito Fiscal), 32 (Consumo), 33 (Nota de Débito), 34 (Nota de Crédito). Opcional ampliar a 41 (Compras), 43 (Gastos Menores), 44 (Regímenes Especiales), 45 (Gubernamental), 46 (Exportaciones), 47 (Pagos al Exterior).
- **Ambientes DGII:** `testecf` (pre-pruebas), `certecf` (certificación formal), `ecf` (producción fiscal).

- **Postulación DGII:** trámite externo del contribuyente; el sistema no lo gestiona pero lo bloquea como pre-requisito para enviar.
 - **Usuario Administrador e-CF:** persona física designada formalmente ante DGII para firmar e-CF en nombre del contribuyente.
 - **Rangos e-NCF:** asignados por DGII por contribuyente y tipo; el sistema los administra como tabla `ecf_sequences` CON `range_start` , `range_end` , `next_number` , `status` y `ambiente` .
 - **Conservación:** evidencias XML firmado + respuesta DGII + TrackId + status logs se conservan al menos 10 años per normativa.
-

8. Qué queda fuera del alcance

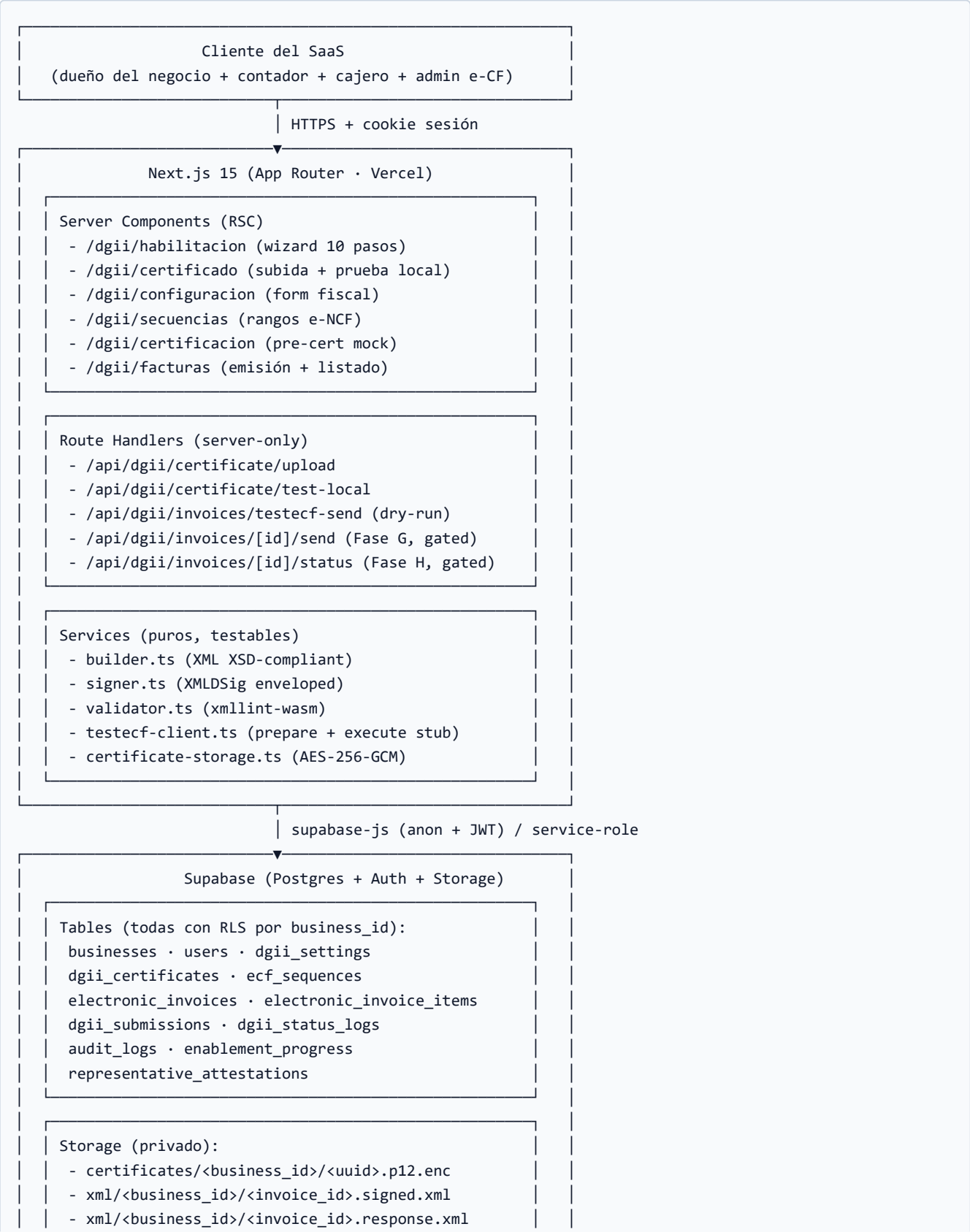
- **No es un sistema contable.** No lleva contabilidad doble entrada ni estado de resultados.
 - **No es un ERP.** No gestiona inventario complejo, compras, RRHH, ni nómina.
 - **No reemplaza al contador.** No emite dictámenes ni asesoría fiscal personalizada.
 - **No emite NCF legados.** Solo e-CF según Norma 06-2018; los NCF físicos pre-2019 quedan fuera.
 - **No recupera certificados perdidos.** Si el cliente pierde el `.p12` o la password, debe comprar uno nuevo a la CA emisora.
 - **No automatiza la postulación DGII.** El cliente / contador la hace en el portal SIRTSS de DGII.
 - **No interfiere con políticas internas del contribuyente.** Quien firma, qué se factura y cuándo es decisión del contador y la administración del cliente.
 - **No es un sustituto del cumplimiento legal.** El cliente sigue siendo responsable ante DGII por errores u omisiones fiscales.
-

9. Principios del sistema

1. **Aislamiento por tenant.** Cada cliente vive bajo su `business_id` ; RLS lo enforza; cross-tenant queries fallan.
2. **Killswitches en capas.** Cada acción potencialmente fiscal está detrás de múltiples flags independientes que deben coincidir.
3. **Dry-run primero, real después.** Cada flow tiene una versión local que se puede ejecutar muchas veces sin consecuencias antes de la versión "live".
4. **Certificado server-only.** El `.p12` nunca llega al cliente; se descifra solo en server actions / route handlers / edge functions.
5. **No secretos en logs.** Passwords, tokens, private keys jamás se imprimen. Tests con spies que detectan leaks.
6. **Errores DGII transparentes.** Lo que DGII devuelve se muestra como vino, sin reinterpretar; el cliente y su contador deciden.
7. **Producción intacta por default.** Vercel Production env queda vacío; el cliente solo activa producción fiscal cuando todo está verde.

8. **Onboarding asistido pero no asumido.** El wizard guía, no asume; cada confirmación requiere acción explícita.
 9. **Documentación al lado del código.** Cada fase deja su runbook, su QA, su checklist de cierre.
 10. **Reutilizable entre SaaS.** El módulo está empaquetado de forma que se pueda copiar a otro proyecto con ajustes mínimos.
 11. **Tests prohíben red.** Suite local nunca golpea DGII real; spy global de `fetch` falla los tests si alguien lo intenta.
 12. **Lo que afecta cumplimiento, se audita.** Todo cambio fiscal se logea en `audit_logs` con `business_id` + `user_id` + timestamp + metadata.
-

10. Arquitectura SaaS multi-tenant recomendada





Capas:

- **UI** (RSC + client components): wizard, formularios, dashboard.
- **API** (route handlers): endpoints internos del SaaS; nunca expuestos a internet sin auth.
- **Services** (puros TS): builder, signer, validator, client DGII — testables sin DB.
- **Persistence** (Supabase): tablas con RLS, storage cifrado, audit_logs.
- **Integraciones** (HTTP): DGII como única externa relevante; CAs son offline.

11. Modelo de datos mínimo

businesses

Columna	Tipo	Notas
id	uuid PK	gen_random_uuid()
legal_name	text	razón social
rnc	text	9 u 11 dígitos
country	text	"DO"
dgii_enabled	boolean	default false
plan_id	uuid FK	a planes SaaS
status	text	active / suspended / trial

users

Columna	Tipo	Notas
id	uuid PK	linkea a <code>auth.users</code>
business_id	uuid FK NOT NULL	tenant key
role	text	admin / cashier / accountant / support
branch_ids	uuid[]	optional

dgii_settings

Columna	Tipo	Notas
business_id	uuid PK FK	1:1 con business
rnc_emisor	text	denormalizado para validación rápida
razon_social_emisor	text	
direccion_emisor	text	
provincia_codigo	text	
municipio_codigo	text	
correo_emisor	text	
telefono_emisor	text	
ambiente	text	<code>testecf</code> / <code>certecf</code> / <code>ecf</code>
dgii_enabled_real_send	boolean	killswitch nivel tenant

dgii_certificates

Columna	Tipo	Notas
id	uuid PK	
business_id	uuid FK NOT NULL	
alias	text	nombre legible (CN del subject)
subject_dn	text	RFC 2253
issuer_dn	text	
serial_number	text	hex
valid_from	timestampz	
valid_to	timestampz	
pkcs12_storage_bucket	text	
pkcs12_storage_path	text	
pkcs12_encrypted_blob	bytea	sealed AES-256-GCM JSON
password_secret_ref	text	sealed AES-256-GCM JSON
kdf	text	"AES-256-GCM"
is_active	boolean	una sola fila activa por business
uploaded_by	uuid FK users	
created_at	timestampz	
revoked_at	timestampz	nullable

Constraint: parcial unique (business_id) WHERE is_active = true .

ecf_sequences

Columna	Tipo	Notas
id	uuid PK	
business_id	uuid FK NOT NULL	
tipo_ecf	text	31/32/33/34/41-47
ambiente	text	testecf/certecf/ecf
range_start	bigint	inicio del rango DGII
range_end	bigint	fin del rango DGII
next_number	bigint	siguiente a reservar
expires_at	timestampz	vencimiento del rango
status	text	active/exhausted/expired/revoked

Función SQL `reserve_next_encf(business_id, tipo, ambiente)` atómica con `SELECT ... FOR UPDATE` .

electronic_invoices

Columna	Tipo	Notas
id	uuid PK	
business_id	uuid FK NOT NULL	
tipo_ecf	text	
e_ncf	text	reservado de ecf_sequences
secuencia_id	uuid FK ecf_sequences	
status	text	draft → generated → validated → signed → submitted → in_process → accepted / accepted_conditional / rejected / cancelled / error
customer_rnc	text	optional
subtotal_gravado	numeric(14,2)	
total_itbis	numeric(14,2)	
total	numeric(14,2)	
xml_generated_path	text	Storage
xml_signed_path	text	Storage
xml_response_path	text	Storage
track_id	text	DGII
dgii_status_code	text	
dgii_status_message	text	
ambiente	text	
generated_at/signed_at/sent_at/accepted_at	timestampz	

electronic_invoice_items

Snapshot inmutable de las líneas. `business_id`, `electronic_invoice_id`, `line_no`, `name_item`, `quantity`, `unit_price`, `itbis_rate`, `monto_item`.

dgii_submissions

Cada envío a DGII (puede haber varios reintentos por invoice).

Columna	Tipo	Notas
id	uuid PK	
business_id	uuid FK	
electronic_invoice_id	uuid FK	
attempt_no	int	
endpoint_url	text	el URL exacto invocado
request_headers	jsonb	sin secretos
request_body_path	text	Storage
response_status	int	HTTP
response_body_path	text	Storage
track_id	text	
error_code / error_message	text	
sent_at / responded_at	timestampz	

dgii_status_logs

Cada consulta de TrackId.

audit_logs

business_id + user_id + action + entity + entity_id + metadata + created_at . RLS INSERT exige business_id = auth_business_id() AND (user_id IS NULL OR user_id = auth.uid()) .

enablement_progress

Estado del wizard por business + step + checklist con evidencia por ítem + declaration_accepted .

representative_attestations

Snapshot inmutable de las 9 evidencias del paso "Autorización del representante e-CF" con responsable, fecha, ref documental, nota.

12. Separación por business_id

Regla: toda tabla del módulo DGII tiene `business_id uuid NOT NULL REFERENCES businesses(id) ON DELETE CASCADE`. Excepciones:

- `businesses` (raíz).
- `users` (linkea a `auth.users` con `business_id` para tenant context).
- Tablas globales (planes, branding del SaaS) no tienen `business_id`.

Toda PK compuesta o índice debe llevar `business_id` como primera columna. Toda FK que cruza tablas del módulo arrastra `business_id` para que la RLS funcione transitivamente.

13. RLS por tenant

Template aplicado a todas las tablas con `business_id`:

```
ALTER TABLE <tabla> ENABLE ROW LEVEL SECURITY;

CREATE POLICY <tabla>_all ON <tabla>
  FOR ALL
  USING (business_id = auth_business_id())
  WITH CHECK (business_id = auth_business_id());
```

`auth_business_id()` lee el claim del JWT en este orden: `root` → `app_metadata` → `user_metadata`. Esto permite que la app use el JWT del seed user para queries y service-role para jobs administrativos.

Verificación: smoke test con dos seed users (business A y business B); el usuario A no puede SELECT, INSERT, UPDATE ni DELETE filas del business B. El test forma parte del CI.

14. Auditoría por tenant

`audit_logs` tiene dos políticas:

```
-- Lectura: solo tu business.
CREATE POLICY audit_logs_select ON audit_logs
  FOR SELECT USING (business_id = auth_business_id());

-- Insert: solo tu business + tu user (o NULL para acciones del sistema).
CREATE POLICY audit_logs_insert ON audit_logs
  FOR INSERT WITH CHECK (
    business_id = auth_business_id()
    AND (user_id IS NULL OR user_id = auth.uid())
  );
```

Eventos auditables mínimos:

- `dgii_certificate_upload` (subida cert) / `dgii_certificate_revoke` .
- `dgii_settings_update` (cambio config fiscal).
- `dgii_sequence_load` (carga rango e-NCF).
- `dgii_invoice_generate` / `_sign` / `_send` / `_status_check` .
- `dgii_representative_attestation_save` .
- `dgii_environment_switch` (testecf → certecf → ecf).

Retención mínima: **10 años** según conservación fiscal.

15. Seguridad de certificados

- **Subida:** form `multipart/form-data` con `.p12` + password. Backend valida MIME, tamaño (< 10KB típico), parsea con `node-forge` , extrae metadata (subject, issuer, vigencia).
 - **Cifrado:** blob `.p12` y password se cifran con **AES-256-GCM**; key de cifrado en env var `DGII_CERT_ENCRYPTION_KEY` (32 bytes random base64) o KMS. IV 12 bytes random por blob; tag 16 bytes; sealed JSON `{v:1, alg:"AES-256-GCM", data:base64(iv|tag|cipher)}` .
 - **Persistencia:** blob cifrado en Supabase Storage privado `certificates/<business_id>/<uuid>.p12.enc` . Solo `service_role` lee.
 - **Descifrado:** solo en server-side (route handlers / server actions / edge functions). El cert PEM y la private key existen en memoria durante el flow de firma; al salir del scope se liberan.
 - **Cliente:** la UI nunca recibe el cert ni la password. Solo recibe metadata pública (subject, vigencia, fingerprint) y resultados de la prueba de firma.
 - **Rotación:** cliente sube nuevo cert → se cifra → se marca `is_active=true` y se desactiva el anterior (`is_active=false`, `revoked_at=now()`). Constraint parcial unique garantiza una sola fila activa.
-

16. Seguridad de secretos

- **Env vars sensibles:** marcadas `--sensitive` en Vercel. Lista mínima: `DGII_CERT_ENCRYPTION_KEY` , `SUPABASE_SERVICE_ROLE_KEY` , `JWT_SECRET` , `DGII_TESTECF_SEND_ENABLED` .
 - **No-go list para Git:** `.env` , `.env.local` , `.vercel/` , `.mcp.json` , `.claude/` , `*.p12` , `*.pfx` , `*.key` , `*.pem` , `*.crt` , `*.cer` , `*.der` , `*.cert` , `certificates/` . `.gitignore` los cubre todos.
 - **Rotación:** `DGII_CERT_ENCRYPTION_KEY` no se rota fácil (requiere re-cifrar todos los blobs); documentar procedimiento. Service role key se rota semestral.
 - **Backups:** key principal en Vault físico + copia en KMS. Procedimiento de recuperación documentado.
 - **Logs:** ningún log de producción puede contener passwords, JWTs, private keys ni contenido cifrado. Tests con regex anti-leak.
-

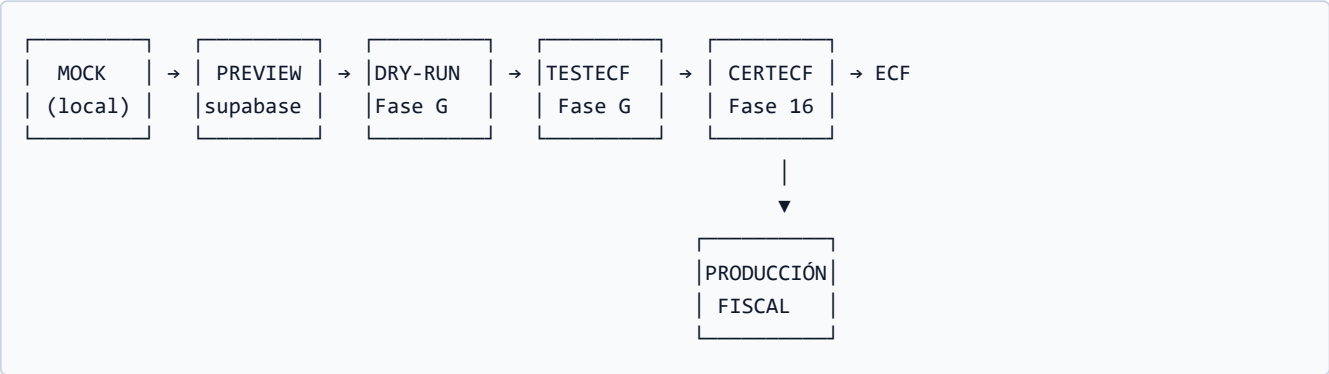
17. Seguridad de logs

- **Server logs:** filtran tokens DGII, passwords, JWTs. Nunca imprimir `request_headers` completos (los headers Authorization se redactan).
 - **Audit logs:** sólo metadata pública (subject DN, fingerprint short, validity, RNC). El `metadata jsonb` valida con un allowlist antes de insertarse.
 - **Client logs:** zero — el frontend no loggea info sensible al console; en producción los console.log se strippean.
 - **Error reporting:** Sentry o equivalente filtra PII (`beforeSend` con scrubbing de RNC, cédula, número de tarjeta).
 - **Logs de DGII:** request/response bodies se guardan en Storage cifrado (path en `dgii_submissions.request_body_path`) — no en logs de aplicación.
-

18. Flujo completo del cliente SaaS

1. Crear negocio
 - └ Signup → onboarding inicial → business creado con plan trial.
2. Configurar datos fiscales
 - └ /dgii/configuracion: RNC, razón social, dirección, provincia/municipio, contacto, ambiente=testecf.
3. Subir certificado
 - └ /dgii/certificado: drag&drop .p12 + password. Backend cifra y persiste.
4. Validar certificado
 - └ Click "Ejecutar prueba local": cargar, vigencia, build XML demo, firma RSA-SHA256, verificar, validar XSD oficial, generar QR demo.
8 steps verdes = cert técnicamente OK para firma.
5. Cargar secuencias
 - └ /dgii/secuencias: capturar rangos e-NCF que DGII asignó al RNC por tipo.
6. Completar checklist
 - └ /dgii/habilitacion paso 8: 9 ítems con evidencia (titular, cédula, RNC emisor, relación, designación, entidad cert, vigencia, CRL, acta) + declaración formal del responsable.
7. Ejecutar dry-run
 - └ Botón "Verificar pre-flight" en wizard. POST /api/dgii/invoices/testecf-send {tipoEcf: "31"}. Backend prepara payload completo, muestra URLs, valida XSD + firma, lista razones de bloqueo del envío real. NO toca DGII.
8. Enviar testecf
 - └ Cuando postulación + rango aprobados, y DGII_TESTECF_SEND_ENABLED=true: botón habilitado. POST /api/dgii/invoices/[id]/send {live:true, postulacionApproved:true, rangoAuthorized:true, userConfirmedAt}. Backend ejecuta auth flow + recepción; guarda track_id.
9. Consultar TrackId/status
 - └ Polling cron Edge Function cada N min consulta cada track_id con status='submitted'|'in_process'. Actualiza electronic_invoices.
10. Certificarse
 - └ Cuando 7-30 días de envíos testecf OK + métricas verdes, cliente solicita transición a certecf en portal DGII; sube evidencia; SaaS cambia ambiente='certecf'.
11. Activar producción fiscal
 - └ Cuando DGII certifica formalmente al contribuyente, el cliente (con autorización admin + contador + dueño SaaS) cambia a ambiente='ecf' y dgii_enabled_real_send=true. Producción fiscal activa.

19. Flujo completo técnico



Cada flecha es **una autorización explícita**. No se salta una etapa. No hay fallback automático de regreso a una etapa superior; cada degradación es manual.

20. Dependencias externas

Dependencia	Rol	Disponibilidad	Plan B
DGII (<code>ecf.dgii.gov.do</code>)	endpoint final de e-CF	SLA no público; outages reales	Cola interna + retry con backoff; comunicar a clientes
Supabase	DB + Auth + Storage	99.9% SLA	Backups diarios; multi-region opcional
Vercel	hosting + edge	99.99% SLA	Branch deploy + rollback < 1 min
Entidad certificadora (Viafirma, Avansi, Certi-Empresa)	emite el cert	Off-system	Cliente compra con anticipación
Contador / asesor fiscal	valida postulación + designación	Cliente humano	Sin él el cliente no certifica
Cliente SaaS	toma decisiones fiscales	Crítico	Soporte asistido con runbook

21. Reglas de seguridad obligatorias

- **NUNCA** llamar a `ecf.dgii.gov.do/ecf/*` desde Preview ni desde tests.
- **NUNCA** subir `.p12` , `.pfx` , `.key` , `.pem` , `.crt` , `.cer` , `.der` , `.cert` a Git.
- **NUNCA** imprimir passwords, JWTs, service role keys, contenido cifrado en logs ni en chat ni en commits.

- **NUNCA** activar `DGII_TESTECF_SEND_ENABLED=true` en Production env hasta que la certificación esté completa.
- **NUNCA** hacer `vercel deploy --prod` sin checklist go-live cumplido.
- **NUNCA** modificar `.env.local`, `.mcp.json`, `.claude/`, ni archivos de certificado en el repo.
- **NUNCA** asumir que la postulación o el rango e-NCF están listos sin confirmación externa documentada.
- **NUNCA** desactivar RLS, ni siquiera "temporalmente para debugging".
- **NUNCA** usar `service_role` desde el cliente; solo desde Edge Functions / Server Actions.
- **NUNCA** hacer fallback automático testecf → certecf → ecf; toda transición es manual y autorizada.

22. Auditoría

Cada acción del módulo escribe en `audit_logs`:

```
await audit.insert({
  business_id: session.businessId,
  user_id: session.user.id,
  user_name: session.user.fullName,
  action: 'dgii_invoice_send',
  entity: 'electronic_invoices',
  entity_id: invoice.id,
  metadata: {
    tipo_ecf: '31',
    track_id: track.id,
    dgii_response_status: 200,
    ambiente: 'testecf',
  },
});
```

Retención: 10+ años. Política de backups asegura inmutabilidad.

Exportación: `/admin/auditoria/export?from=YYYY-MM-DD&to=YYYY-MM-DD` genera CSV/JSON filtrado por `business_id` (RLS aplica).

Consulta: UI dedicada con filtros por acción, entidad, usuario, rango de fechas.

23. Manejo de errores DGII

Códigos comunes y respuesta esperada:

Código DGII	Significado	Acción del sistema
200	Aceptado / TrackId emitido	Guardar <code>track_id</code> , status <code>submitted</code>
400	Error de schema o body malformado	Marcar <code>electronic_invoices.status='error'</code> , exponer mensaje al usuario
401	Token inválido o expirado	Re-ejecutar Semilla → ValidarSemilla; reintento limitado
403	Contribuyente no autorizado o ambiente incorrecto	Status <code>error</code> , alarma a contador (postulación pendiente?)
404	Endpoint no existe	Verificar <code>DGII_BASE_URL_*</code> ; alarma a devops
429	Rate limit	Backoff exponencial (1m, 5m, 15m, 30m)
5xx	DGII caída	Cola interna <code>status='pending_retry'</code> ; reintento con backoff; alarma
Timeout	Sin respuesta en 30s	Reintento; si persiste, alarma DGII outage

Retry policy:

- Reintentos automáticos: 3 con backoff (1m, 5m, 15m).
- Después de 3 fallidos: status `error` + alarma manual.
- Rate limit por business: máx 60 envíos/min al inicio.

Alarmas:

- Tasa de rechazo > 5% en 1h → notificar a soporte + cliente.
 - DGII timeout > 3 consecutivos → notificar oncall.
 - Cert vence en < 30 días → notificar cliente y contador.
-

24. Riesgos críticos

#	Riesgo	Probabilidad	Impacto	Mitigación	Dueño
R-01	Pérdida de <code>DGII_CERT_ENCRYPTION_KEY</code>	Baja	Crítico	Backup en Vault físico + KMS; procedimiento documentado	Seguridad
R-02	Cert vencido sin aviso	Media	Alto	Cron alerta 60/30/7 días antes	Soporte + cliente
R-03	Cross-tenant data leak	Baja	Crítico	RLS en todas las tablas + tests automatizados	Backend
R-04	Envío masivo accidental a DGII	Baja	Alto	<code>dgii_enabled_real_send</code> per-business + rate limit + killswitches	DevOps
R-05	Secuencia agotada en mid-venta	Media	Alto	Validación <code>next_number <= range_end - 100</code> antes de cada emisión; alarma 90% del rango	Backend
R-06	XML rechazado por XSD post-deploy	Media	Alto	Validar contra XSD oficial antes de cada envío; tests del builder	Backend
R-07	DGII outage prolongado	Baja-Media	Alto	Cola <code>pending_retry</code> ; UI muestra estado "DGII caída"	DevOps + soporte
R-08	Datos del emisor incorrectos en XML	Baja	Alto	Validar <code>dgii_settings.rncEmisor</code> contra SIRTSS al guardar	Compliance
R-09	Mismatch RNC cert ↔ business	Baja	Alto	Validación al activar cert; bloquear si mismatch	Compliance
R-10	Filtración del cert / password	Muy baja	Crítico	AES-256-GCM + Storage privado + decrypt solo server	Seguridad
R-11	Re-uso de eNCF (duplicate)	Muy baja	Crítico	Función SQL atómica <code>reserve_next_encf</code> con FOR UPDATE	Backend
R-12	Token DGII filtrado en logs	Baja	Alto	Redactor de headers; tests anti-leak	Seguridad
R-13	Vercel Production env contaminado	Baja	Crítico	Política "cero env vars en Production hasta certificación"; revisión per-cambio	DevOps
R-14	Postulación con datos viejos	Media	Alto	Validar antes de cada activación testecf; checklist § 3.7	Contador
R-15	Cliente cambia razón social y emite con dato viejo	Baja	Alto	Snapshot inmutable de datos del emisor por invoice	Backend

25. Decisiones técnicas recomendadas

1. **XMLDSig enveloped (no XAdES-BES)** según documento oficial DGII; `SignatureMethod=rsa-sha256`, `DigestMethod=sha256`, `CanonicalizationMethod=c14n-20010315`, `Reference URI=""` con `isEmptyUri:true` para que xml-crypto no añada `Id` al elemento ECF (el XSD no lo permite).
 2. **multipart/form-data** para recepción ECF según la doc oficial DGII.
 3. **eNCF format:** `^[A-Z][0-9]{12}$` — primera letra E, dos siguientes el tipo (31/32/33/34), 10 dígitos del rango.
 4. `xmllint-wasm` **para validación XSD** — sin dependencias nativas, funciona en Vercel functions; alternativa `libxmljs2` rota el build de Vercel.
 5. `xml-crypto v6+` para XMLDSig; antes de v6 tenía bugs con Reference URI vacío.
 6. `node-forge` para PKCS#12 — más portable que `openssl` shell.
 7. **Encoding del XML:** UTF-8 declarado; sin BOM.
 8. **Storage cifrado server-side** para blobs `.p12` — Supabase Storage privado con `service_role`; Edge Function descifra al firmar.
 9. `DGII_TESTECF_SEND_ENABLED` como killswitch independiente de los flags per-business — capa adicional.
 10. **Tests prohíben fetch real** — `vi.spyOn(global, 'fetch')` que tira; verificación al final del suite.
-

26. Checklist antes de producción

- ☐ Cert vigente > 90 días.
- ☐ Postulación DGII aprobada formalmente.
- ☐ Rangos e-NCF de producción autorizados por DGII para cada tipo emitible.
- ☐ Margen suficiente: `next_number <= range_end - 100`.
- ☐ Backup Supabase del día (verificado restorable).
- ☐ Cron polling status Fase H activo y monitoreado.
- ☐ Alarmas configuradas: rechazo > 5%, DGII timeout > 3, secuencia 90% agotada, cert vence en 30 días.
- ☐ Plan de rollback documentado y simulado en stage.
- ☐ Acta de habilitación DGII recibida y archivada.
- ☐ Contador firmó declaración jurada interna.
- ☐ Comunicación a clientes: emisión activa, política de e-CF.
- ☐ Soporte capacitado para errores comunes.
- ☐ Runbook actualizado para "qué hacer si DGII está caída".
- ☐ Acceso a logs de auditoría verificado por auditor.
- ☐ Política de retención de evidencias (10 años) implementada.

Solo cuando los 15 items estén verdes: `ambiente='ecf'` y `dgii_enabled_real_send=true` per business autorizado.

27. Estrategia de soporte

- **Canales:** chat in-app, email, ticket. Soporte solo desde rol "support" con acceso de solo-lectura a `audit_logs` y `dgii_submissions`.
 - **SLA:**
 - P0 (DGII outage o cliente no puede emitir): respuesta < 15 min, mitigación < 1h.
 - P1 (cliente bloqueado por config): respuesta < 1h, resolución < 4h.
 - P2 (consulta general): respuesta < 4h, resolución < 24h.
 - **Runbook:** `docs/dgii/runbook-soporte.md` (referencia a `runbook-fase-f-g-h.md` y a este plan maestro).
 - **Escalation:** soporte → ingeniero backend → líder técnico → compliance/contador.
 - **Capacitación:** mensual sobre cambios DGII, casos comunes, lectura de XML.
-

28. Estrategia de onboarding de clientes

5 etapas:

1. **Signup + provisioning** (día 0): cliente firma, recibe acceso al SaaS, business creado.
2. **Habilitación** (día 0-7): wizard `/dgii/habilitacion` — paso 1 a 10. Soporte acompaña por chat.
3. **Pre-Fase G** (día 7-14): checklist representante con contador + obtener cert + cargar secuencias mock.
4. **Testecf** (día 14-21): enviar 10+ comprobantes de prueba a DGII testecf; resolver rechazos.
5. **Certecf → producción** (día 21-45): solicitar transición DGII, certificación formal, activar producción fiscal.

Métricas:

- TTHabilitación (signup → wizard 100%): < 7 días media.
 - TTProducción (signup → emite real): < 45 días media.
 - Sin-soporte rate: % de clientes que completan habilitación sin ticket > 60%.
-

29. Estrategia de monitoreo

Alarmas críticas (página oncall):

- DGII responde 5xx > 3 consecutivos.
- Rechazo > 5% en 1h.

- Cert vencido en < 7 días (per business).
- Secuencia agotada (`next_number > range_end`) per business + tipo.
- Audit logs no se insertan (RLS issue).
- Spike de errores 401 (token o postulación).

Dashboards:

- Volumen de e-CFs emitidos por día / tipo / business.
- Tasa de aceptación DGII por business.
- Latencia p95 del flow `submit` .
- Edge Function ejecución y errores.
- Backups Supabase status.

Métricas de negocio:

- Clientes en cada estado del enablement.
- Tiempo medio para llegar a `ready_for_testecf` .
- % de clientes que llegan a producción fiscal.

30. Estrategia de recuperación ante errores

- **Backups Supabase:** point-in-time recovery 7 días + snapshot diario retenido 30 días + snapshot mensual retenido 1 año + snapshot anual retenido 10 años (compliance fiscal).
- **Rollback de fase:** cada fase del roadmap tiene rollback documentado. Ej. revertir Fase G se hace con `UPDATE dgii_settings SET dgii_enabled_real_send=false` y desactivando el flag de env.
- **DGII outage fallback:** cola `pending_retry` ; UI muestra banner "DGII caída"; el cliente sigue operando offline; al volver DGII, los envíos drenan automáticamente.
- **Cert revocado de emergencia:** `UPDATE dgii_certificates SET is_active=false, revoked_at=now()` ; UI vuelve a "Sin certificado activo".
- **Recuperación de `DGII_CERT_ENCRYPTION_KEY`** : procedimiento de 2 personas + Vault físico; sin la key los certs son irrecuperables (se vuelve a subir cada uno).

31. Estrategia para múltiples clientes SaaS

Tenant lifecycle:

- Trial (30 días) → Active → Suspended (impago) → Cancelled (datos retenidos 30 días) → Hard-delete (cumpliendo retención fiscal de 10 años para evidencias).
- Suspension: bloquea emisión real DGII pero conserva datos.
- Hard-delete: `audit_logs` y `electronic_invoices` se exportan a archivo cifrado para retención legal antes de borrar.

Billing:

- Por business + plan. e-CFs emitidos como métrica secundaria.
- Métricas para upselling: % uso de rango eNCF, # comprobantes/mes.

Multi-region:

- Supabase región us-east-1 (cercano a DGII en RD).
- Vercel edge en mismo región para latencia.

32. Cómo adaptar el módulo a otro sistema

Reutilizable tal cual:

- `apps/web/src/server/services/dgii/*` (builder, signer, validator, testecf-client, preflight).
- `supabase/migrations/000[1-7]_*.sql` (multi-tenant + DGII).
- `docs/dgii/xsd/*.xsd` (XSDs oficiales).
- Tests del builder/signer/validator.

Adaptar al nuevo proyecto:

- UI (`/dgii/habilitacion` , `/dgii/certificado` , etc.) — copiar y ajustar al sistema de design / componentes del nuevo SaaS.
- `env.ts` schema — adaptar a las env vars del nuevo proyecto.
- `getSession()` — reemplazar por el auth del nuevo proyecto.
- `getRepositories()` — adaptar al data layer existente.

Dejar en el nuevo proyecto:

- Branding (logo, colors).
- Modelos de negocio específicos (productos, pricing).
- Integraciones específicas (Whatsapp, OpenAI, etc.).

Plan de migración recomendado:

1. Importar las migraciones Supabase.
2. Importar los services puros (sin dependencias del proyecto original).
3. Instalar deps (`node-forge` , `xml-crypto` , `xmllint-wasm` , `xmlbuilder2`).
4. Crear las rutas UI con tu sistema de design.
5. Adaptar el wizard a tu lenguaje / tono.
6. Correr el QA SaaS pre-Fase G como smoke test.

33. Lecciones aprendidas de DermaLand

25 lecciones extraídas de la implementación real:

1. **Separar mock, preview y producción** desde día 1 — `DATA_SOURCE` env y `dgii_enabled_real_send` permiten emitir mock indefinidamente.

2. **Producción puede estar en MOCK/DEMO mientras se certifica** — no hay que esperar a tener cert real para deployar; el feature flag protege.
3. **Nunca tocar DGII real antes de tiempo** — incluso GET /Semilla deja huella en logs DGII y puede confundir al contador.
4. **Validar certificado localmente antes de testecf** — prueba local con XSD oficial confirma que el cert puede firmar XML válido; ahorra una iteración con DGII.
5. **No enviar a testecf sin postulación** — DGII responde 401 con mensaje opaco; mejor bloquear en cliente.
6. **No enviar a testecf sin rango e-NCF** — incluso con postulación, sin rango asignado el comprobante se rechaza.
7. **Usar dry-run antes de POST real** — `prepareTestecfSubmission` permite ver el payload exacto sin riesgo.
8. **Guardar certificado cifrado** — nunca en texto plano, nunca en filesystem del server, nunca en repo.
9. **Nunca imprimir secretos** — tests con `vi.spyOn(global, 'fetch')` y regex en evidencias garantizan que un cambio no introduzca leak.
10. **No subir `.p12` / `.pfx` / `.key` / `.pem` a Git** — `.gitignore` exhaustivo + pre-commit hook + revisión de PRs.
11. **RLS por `business_id`** — no es opcional; cualquier tabla que no tenga RLS es un agujero.
12. **Auditoría por `business_id`** — `audit_logs` requiere INSERT policy con `user_id = auth.uid()` para evitar suplantación.
13. **Fase G debe ser autorizada por cliente y por tipo e-CF** — no autorización genérica; cada tipo es un trámite separado.
14. **Cada tenant debe tener su propio certificado** — no hay "cert del SaaS"; el cert es del contribuyente.
15. **Cada tenant debe tener sus propias secuencias** — DGII las asigna por RNC.
16. **Cada tenant debe tener su configuración fiscal** — `dgii_settings` 1:1 con business.
17. **Cada tenant debe tener su evidencia** — `representative_attestations` con responsable + fecha + ref doc por ítem.
18. **Cada tenant debe tener auditoría separada** — RLS aplicada también a `audit_logs`.
19. **El certificado puede ser de persona física** — DGII lo exige así para procesos tributarios; validar titularidad como Usuario Administrador e-CF en checklist § 3.7.
20. **La validación técnica no sustituye la aprobación DGII** — XSD verde + firma OK no significa que DGII vaya a aceptar; postulación + rango siguen siendo necesarios.
21. **El cliente/contador debe validar postulación y rangos** — el sistema no puede; los muestra como gates explícitos.
22. **No hacer fallback automático testecf → certecf → producción** — toda transición es manual con autorización per-cliente.
23. **Mantener killswitches** — `DGII_TESTECF_SEND_ENABLED` default false; `dgii_enabled_real_send` default false; `ambiente=testecf` default.
24. **Mantener botones de envío real deshabilitados hasta completar requisitos** — incluso con todo verde, el botón sigue disabled hasta que el flag explícito esté activo.

25. **Registrar todo lo que afecta cumplimiento fiscal** — cada decisión, cambio, error, retry, status check va al `audit_logs` .

34. Glosario

Término	Significado
e-CF	Comprobante Fiscal Electrónico — XML firmado emitido al ambiente DGII.
NCF	Número de Comprobante Fiscal — formato legado anterior a 2019.
eNCF	Electronic NCF — secuencia 13 chars <code>E + tipo(2) + numero(10)</code> (ej. <code>E310000000001</code>).
RNC	Registro Nacional del Contribuyente — 9 dígitos (jurídicas) u 11 (físicas).
testecf	Ambiente DGII de pre-pruebas. URL base <code>https://ecf.dgii.gov.do/testecf/</code> .
certecf	Ambiente DGII de certificación formal. URL base <code>https://ecf.dgii.gov.do/certecf/</code> .
ecf	Ambiente DGII de producción fiscal real. URL base <code>https://ecf.dgii.gov.do/ecf/</code> .
Postulación	Trámite del contribuyente en DGII para habilitar e-CF; entrega documentos + cert + URLs.
Usuario Administrador e-CF	Persona física designada formalmente ante DGII para firmar e-CF a nombre del contribuyente.
XAdES-BES	XML Advanced Electronic Signature - Basic Electronic Signature (no usado por DGII actualmente).
XMLDSig	XML Digital Signature — estándar W3C; usado por DGII (enveloped, c14n, RSA-SHA256).
TrackId	Identificador devuelto por DGII al aceptar un envío; usado para consultar status.
Semilla	XML con un valor aleatorio que DGII devuelve para autenticación; el cliente lo firma y lo retorna.
ValidarSemilla	Endpoint DGII que recibe la semilla firmada y devuelve bearer token.
RLS	Row-Level Security — Postgres feature para filtrar filas por usuario/tenant.
PKCS#12	Formato estándar de contenedor de cert + private key + chain (extensión <code>.p12</code> o <code>.pfx</code>).
AES-256-GCM	Algoritmo de cifrado autenticado simétrico con clave 256 bits.
CRL	Certificate Revocation List — lista de certs revocados por la CA emisora.
OCSP	Online Certificate Status Protocol — query online para verificar si un cert está revocado.
CA	Certificate Authority — entidad que emite certs (Viafirma, Avansi, Certi-Empresa, etc.).
INDOTEL	Instituto Dominicano de Telecomunicaciones — regula las CAs en RD.
SIRTSS	Sistema DGII para trámites del contribuyente; donde se hace la postulación.
Norma 06-2018	Norma DGII que regula la facturación electrónica.
Killswitch	Flag de alto nivel que bloquea una funcionalidad crítica con efecto inmediato.

Término	Significado
Dry-run	Ejecución que prepara todo sin hacer el efecto final (ej. sin tocar DGII).
Pre-flight	Verificación previa al envío real (URLs, validaciones, gates).
Tenant	Cliente del SaaS; en el modelo equivale a un <code>business_id</code> .
Multi-tenant	Arquitectura donde múltiples tenants comparten infra con aislamiento.
Wizard de habilitación	Flujo guiado de 10 pasos para que el cliente complete el proceso DGII.
Ambiente	Uno de los tres entornos DGII: testecf / certecf / ecf.

35. Anexos

A. URLs DGII públicas oficiales

- testecf: `https://ecf.dgii.gov.do/testecf/`
- certecf: `https://ecf.dgii.gov.do/certecf/`
- ecf: `https://ecf.dgii.gov.do/ecf/`
- Portal contribuyente SIRTSS: `https://dgii.gov.do/cicloContribuyente/`
- Documentación técnica e-CF (referencia, no enlace directo a cambios): página oficial DGII / sección Comprobantes Electrónicos.

B. Documentos complementarios del paquete

- `prd-saas-facturacion-electronica-dgii.md` — PRD.
- `agentes-y-skills-saas-dgii.md` — agentes + skills + prompts.
- `roadmap-fases-saas-dgii.md` — 20 fases.
- `checklist-implementacion-saas-dgii.md` — checklists.
- `plan-maestro-saas-dgii-completo.pdf` — consolidado.

C. Documentos internos relevantes en DermaLand (referencia)

- `docs/dgii/runbook-fase-f-g-h.md` — operativo de Fases F/G/H.
- `docs/dgii/qa-saas-pre-fase-g.md` — checklist QA manual.
- `docs/dgii/matriz-requisitos-dgii.md` — matriz C-XX de requisitos.
- `docs/dgii/requisitos-facturacion-electronica-dgii.md` — fuente normativa.
- `PROJECT_MEMORY.md` — historial de sesiones.

D. Plantillas

- Plantilla de checklist por ítem (ver Doc 5).

- Plantilla de prompt por agente (ver Doc 3).
- Plantilla de fase del roadmap (ver Doc 4).

E. Versionado

Versión	Fecha	Cambios
1.0	2026-05-21	Documento inicial basado en DermaLand pre-Fase G real.

Fin del Plan Maestro. Continuar con los Docs 2-5 del paquete para profundizar en cada dimensión.

Capítulo 2 — PRD / Product Requirements

PRD — SaaS de Facturación Electrónica DGII para República Dominicana

Versión: 1.0 · **Fecha:** 2026-05-21 **Estado:** PRD profesional, base para implementación **Audiencia:** Producto, ingeniería, contador/asesor fiscal, dueño SaaS, soporte, auditor

1. Nombre del producto

SaaS DGII e-CF para RD — módulo reusable de facturación electrónica que cualquier SaaS dominicano puede integrar para que sus clientes emitan e-CFs con cumplimiento DGII.

Primera instancia de referencia: DermaLand (SaaS farmacia/dermocosmética).

2. Problema

Los SaaS RD que necesitan ofrecer facturación electrónica a sus clientes enfrentan tres problemas simultáneos:

- **Técnico:** XMLDSig, XSD oficial, criptografía PKCS#12, integración HTTP con DGII — todas piezas con potencial de fallo silencioso.
- **Fiscal:** Norma DGII 06-2018, postulación, Usuario Administrador e-CF, rangos e-NCF, ambientes — requieren coordinación con contador y DGII.
- **Operativo:** onboarding del cliente final no-técnico, soporte, conservación 10 años, manejo de errores DGII.

Implementar esto desde cero por proyecto duplica esfuerzo y multiplica riesgo de errores que comprometen cumplimiento fiscal.

3. Oportunidad

Un módulo reusable bien encapsulado permite:

- Integrar e-CF en un SaaS RD en semanas (no meses).
 - Onboarding asistido del cliente sin requerir equipo técnico dedicado.
 - Compliance fiscal por diseño con killswitches en capas.
 - Auditoría completa con retención 10 años.
 - Costos compartidos en mantenimiento de XSDs, normativa, endpoints DGII.
-

4. Usuarios objetivo

4.1 Dueño del negocio

Responsable del cumplimiento fiscal del contribuyente. No es técnico. Necesita: confianza en que el sistema "no me va a meter en lío con DGII", reportes claros, facilidad para delegar al contador.

4.2 Contador / asesor fiscal

Validación fiscal de cada etapa: postulación, designación Usuario Administrador e-CF, configuración de tasas ITBIS, rangos e-NCF. Necesita: visibilidad de la configuración fiscal, exportación de evidencias, capacidad de auditar audit_logs.

4.3 Administrador e-CF

Persona física designada formalmente ante DGII para firmar e-CFs. Coincide con el titular del certificado. Necesita: claridad sobre qué se firma con su nombre, capacidad de revocar el cert si pierde la designación.

4.4 Cajero

Emite comprobantes desde el POS. No le interesa el detalle DGII. Necesita: emisión rápida, mensajes claros si falla, fallback simple cuando DGII está caída.

4.5 Soporte SaaS

Acompaña al cliente durante onboarding y operación. Necesita: acceso de solo-lectura a logs por business, runbook claro, escalation matrix.

4.6 Equipo técnico SaaS

Mantenimiento del módulo. Necesita: tests automatizados, documentación, ambientes claros, deploy preview, rollback simple.

4.7 Auditor

Verifica cumplimiento fiscal post-hoc. Necesita: acceso completo a audit_logs por business (solo-lectura), exportación inmutable, conservación 10 años.

5. Jobs to be done

Por dueño del negocio

1. "Cuando llegue mi fecha de obligatoriedad DGII, quiero estar emitiendo e-CFs sin haber tocado código."
2. "Cuando un cliente me pida una factura, quiero darle un e-CF válido en menos de 30 segundos."

3. "Cuando DGII haga una fiscalización, quiero entregar toda la evidencia exportada en una hora."

Por contador

1. "Cuando el dueño me consulte sobre configuración fiscal, quiero ver y validar todo desde una pantalla."
2. "Cuando un e-CF sea rechazado por DGII, quiero ver el mensaje exacto de DGII para diagnosticar."
3. "Cuando llegue auditoría, quiero exportar todos los XMLs firmados + respuestas DGII del período."

Por administrador e-CF

1. "Cuando se firme un e-CF con mi cert, quiero que quede registro auditable de a qué hora y por qué."
2. "Cuando pierda la designación, quiero revocar mi cert con un click."

Por cajero

1. "Cuando emita un comprobante, quiero el ticket impreso con el QR DGII listo."
2. "Cuando DGII esté caída, quiero seguir cobrando y que el sistema se encargue después."

Por soporte

1. "Cuando un cliente me llame con un error, quiero ver inmediatamente la última transacción y el error DGII."

6. Objetivos de producto

Objetivo	Métrica	Target
OP-01	Tiempo desde signup hasta primer e-CF emitido (sin asistencia técnica)	< 7 días p50
OP-02	% de clientes que completan habilitación sin abrir ticket	> 60%
OP-03	% de e-CFs aceptados por DGII en primer intento	> 95%
OP-04	Tiempo medio para resolver rechazo DGII	< 4h
OP-05	NPS post-onboarding	> 50

7. Objetivos técnicos

Objetivo	Métrica	Target
OT-01	Disponibilidad del módulo	> 99.5%
OT-02	Latencia p95 del flow <code>submit</code> (sin contar DGII)	< 500ms
OT-03	Cobertura tests builder + signer + validator	> 90%
OT-04	Zero data leak cross-tenant en pruebas	100%
OT-05	Zero secrets en logs / commits / PRs	100%

8. Objetivos fiscales

Objetivo	Métrica	Target
OF-01	Cumplimiento Norma 06-2018	Auditoría externa anual OK
OF-02	Separación testecf / certecf / ecf	Sin contaminación cruzada
OF-03	Sin emisión fiscal accidental	Cero incidentes
OF-04	Conservación evidencias 10 años	Auditable per business
OF-05	RNC emisor coincide con cert al 100%	Verificación pre-envío

9. Métricas de éxito

North Star: % de contribuyentes clientes del SaaS que están emitiendo e-CFs en producción fiscal con tasa de aceptación > 95%, sin haber escalado a soporte técnico nivel 2 durante el último mes.

Secundarias:

- Time-to-first-e-CF (signup → primer envío aceptado en testecf).
- Time-to-production-fiscal (signup → ambiente= `ecf`).
- MTTR para DGII outage (detección → mitigación).
- % de e-CFs aceptados en primer intento.
- Tickets de soporte por cliente / mes.

10. Casos de uso

CU-01 — Cliente nuevo configura su negocio

Actor: Dueño del negocio. **Precondición:** signup completado. **Flujo:** entra a `/dgii/habilitacion` → wizard paso 1 → captura RNC, razón social, ambiente=testecf → guarda config → status pasa de `not_started` a `in_preparation`. **Postcondición:** `dgii_settings` poblada para el business.

CU-02 — Subida de certificado real

Actor: Administrador e-CF. **Precondición:** business con config fiscal completa; tiene `.p12` y password. **Flujo:** `/dgii/certificado` → drag&drop `.p12` + password → backend valida + cifra + persiste → metadata visible. **Postcondición:** `dgii_certificates.is_active=true`, audit log `dgii_certificate_upload`.

CU-03 — Prueba local del certificado

Actor: Admin e-CF. **Precondición:** cert activo. **Flujo:** click "Ejecutar prueba local" → backend descifra → build XML demo → firma → verifica → valida XSD oficial → genera QR demo → devuelve evidencia. **Postcondición:** evidencia persistida; usuario ve 8 steps verdes.

CU-04 — Pre-flight Fase G dry-run

Actor: Dueño / Admin e-CF. **Precondición:** cert activo + config fiscal. **Flujo:** click "Verificar pre-flight" en `/dgii/habilitacion` → POST `/api/dgii/invoices/testecf-send {tipoEcf:31}` → backend prepara payload + URLs + razones de bloqueo → UI muestra resultado. **Postcondición:** ninguna llamada DGII; bloqueos visibles.

CU-05 — Completar autorización del representante

Actor: Dueño + contador. **Precondición:** paso 7 wizard avanzado. **Flujo:** abrir paso 8 → confirmar 9 ítems (titular, cédula, RNC, relación, designación, entidad cert, vigencia, CRL, acta) con responsable + fecha + ref documental + nota → aceptar declaración formal. **Postcondición:** `representative_attestations` con 9 filas; declaración aceptada; gate `ready_for_testecf` desbloqueado.

CU-06 — Envío real a testecf (Fase G)

Actor: Sistema (gated por dueño). **Precondición:** postulación + rango + cert + config + 9 ítems + declaración + `DGII_TESTECF_SEND_ENABLED=true`. **Flujo:** POST `/api/dgii/invoices/[id]/send {live:true,...}` → backend: Semilla → ValidarSemilla → multipart submit → recibe TrackId → persiste `dgii_submissions` + audit log. **Postcondición:** `electronic_invoices.track_id` poblado; status= `submitted`.

CU-07 — Consulta de status (Fase H)

Actor: Cron + cliente. **Precondición:** `electronic_invoices.status IN ('submitted','in_process')`. **Flujo:** Edge Function corre cada N min → GET `/consultatrackid/api/TrackIds/Estado` → mapea respuesta DGII →

actualiza status → INSERT `dgii_status_logs` . **Postcondición:** status actualizado a `accepted` / `accepted_conditional` / `rejected` .

CU-08 — DGII rechaza un e-CF

Actor: Backend + soporte + cliente. **Precondición:** envío realizado, DGII responde `rejected` . **Flujo:** status= `rejected` , mensaje DGII guardado, alarma a soporte. UI muestra al cliente el mensaje exacto + sugerencias. **Postcondición:** cliente entiende qué falla y puede corregir (re-emitir tras fix).

CU-09 — DGII caída

Actor: Backend. **Precondición:** DGII responde 5xx o timeout > 3 consecutivos. **Flujo:** sistema marca DGII en estado "caída", encola envíos pendientes, UI muestra banner, alarma DevOps. Al volver DGII, queue drena automáticamente. **Postcondición:** sin pérdida de envíos; trazabilidad completa.

CU-10 — Auditoría externa solicita evidencia

Actor: Auditor + dueño. **Precondición:** business con histórico. **Flujo:** `/admin/auditoria/export?from=&to=` → CSV/JSON con `audit_logs` + XMLs firmados + respuestas DGII → entregado al auditor. **Postcondición:** evidencia exportada inmutable.

11. User stories

- **US-01:** Como dueño, quiero un wizard guiado para no equivocarme con DGII.
- **US-02:** Como dueño, quiero ver el estado global ("Listo para testecf") para saber dónde estoy.
- **US-03:** Como contador, quiero ver qué configuración fiscal está activa para validarla.
- **US-04:** Como contador, quiero recibir el XML + respuesta DGII de cada e-CF para mi archivo.
- **US-05:** Como admin e-CF, quiero subir mi `.p12` sin que mi password quede en logs.
- **US-06:** Como admin e-CF, quiero ver una prueba local antes de tocar DGII.
- **US-07:** Como cajero, quiero emitir un comprobante en < 30 segundos.
- **US-08:** Como cajero, quiero un mensaje claro si DGII rechaza, sin jerga técnica.
- **US-09:** Como soporte, quiero ver todos los envíos recientes de un cliente filtrados.
- **US-10:** Como soporte, quiero un runbook que cubra los 10 errores más comunes.
- **US-11:** Como dueño, quiero estar seguro de que ningún otro cliente del SaaS ve mis datos.
- **US-12:** Como técnico, quiero tests automatizados que validen RLS cross-tenant.
- **US-13:** Como técnico, quiero killswitches que bloqueen envío real sin autorización.
- **US-14:** Como auditor, quiero acceso de solo-lectura a `audit_logs` por business.
- **US-15:** Como dueño, quiero alertas si mi certificado vence en < 30 días.
- **US-16:** Como dueño, quiero alertas si mi rango e-NCF está cerca de agotarse.
- **US-17:** Como dueño, quiero un botón "Enviar pruebas a DGII testecf" deshabilitado hasta que esté listo.

- **US-18:** Como contador, quiero ver el RNC del cert y el RNC del emisor para confirmar que coinciden.
 - **US-19:** Como técnico, quiero deploy preview sin tocar producción.
 - **US-20:** Como dueño, quiero exportar mi histórico cuando deje de usar el SaaS.
-

12. Requisitos funcionales

#	Requisito	Criterio de aceptación
RF-01	Wizard /dgii/habilitacion con 10 pasos	Carga con 3 banners + 10 cards en orden
RF-02	Subida cert .p12 cifrada server-side	Blob persistido cifrado AES-256-GCM en Storage
RF-03	Prueba local del cert con 8 steps	Todos los steps son verificables; resultado json
RF-04	Validación XSD oficial e-CF 31/32/33/34	XSDs en docs/dgii/xsd/ ; validateEcfxml retorna valid:true para XML correcto
RF-05	Firma XMLDSig enveloped RSA-SHA256	Firma verificable con clave pública; Reference URI vacío
RF-06	Form de evidencia paso 8 (9 ítems)	Cada ítem persiste status + responsable + fecha + ref + nota
RF-07	Declaración formal del responsable	Checkbox + timestamp; bloquea ready_for_testecf si no aceptado
RF-08	Panel "Pendiente antes de enviar a DGII testecf"	Lista gaps dinámicos; badge listo cuando todos verdes
RF-09	CTA "Enviar pruebas a DGII testecf" disabled	Disabled en este wizard; tooltip explica
RF-10	Pre-flight dry-run sin tocar DGII	UI muestra URLs + XSD + firma + bloqueos
RF-11	Envío real testecf (Fase G)	Auth flow + recepción multipart + TrackId guardado
RF-12	Consulta TrackId (Fase H)	Cron polling con rate limit; status actualizado
RF-13	Aislamiento por business_id (RLS)	Tests cross-tenant fallan al insertar/leer ajeno
RF-14	Auditoría INSERT con own user_id	RLS bloquea suplantación
RF-15	Reportes fiscales por período	Filtro fecha + tipo + status; CSV exportable
RF-16	Listado de e-CFs emitidos	Paginado, búsqueda por eNCF / cliente

#	Requisito	Criterio de aceptación
RF-17	Permisos por rol DGII	dgii:configure / cert:upload / sequences:manage / invoices:* asignables
RF-18	Killswitch DGII_TESTECF_SEND_ENABLED	env flag default false
RF-19	Rate limit interno	máx 60 envíos/min por business
RF-20	Notas de crédito (e-CF 34)	Form que referencia comprobante origen + razón
RF-21	PDF de representación impresa con QR	Template con todos los campos requeridos + advertencia NO FISCAL en demo
RF-22	Manejo errores DGII transparente	Mensaje exacto DGII mostrado al usuario
RF-23	Cola pending_retry	Reintentos automáticos con backoff
RF-24	Alerta cert vencimiento	60/30/7 días antes
RF-25	Alerta secuencia agotada	< 100 disponibles
RF-26	Exportación auditoría	CSV/JSON por período
RF-27	Roles support read-only	Solo SELECT en audit_logs y submissions
RF-28	Cambio de ambiente requiere autorización	Confirmación + audit log
RF-29	Backup automático Supabase	Diario + retención 10 años
RF-30	Rollback documentado por fase	Procedimiento testeable

13. Requisitos no funcionales

- **Performance:** p95 pre-flight < 500ms; p95 send (sin DGII) < 1s; consulta status < 300ms.
- **Disponibilidad:** 99.5% mensual del módulo (excluye outages DGII).
- **Seguridad:** OWASP Top 10 cubierto; auditoría externa anual.
- **Escalabilidad:** 10,000 e-CFs/día por business; 1,000 businesses concurrentes.

- **Mantenibilidad:** typecheck verde; tests > 90% cobertura services puros.
 - **Observabilidad:** logs estructurados; métricas por business; dashboards.
 - **Compatibilidad:** Chrome, Firefox, Edge, Safari últimas 2 versiones; mobile responsive.
 - **Idioma:** español RD por default; código en inglés (variables, funciones).
-

14. Requisitos legales / fiscales

- **DGII Norma 06-2018** y actualizaciones vigentes.
 - **Postulación oficial** del contribuyente en testecf → certecf → ecf.
 - **Usuario Administrador e-CF** designado en SIRTSS DGII.
 - **Vigencia del cert** > 60 días para emitir; renovación documentada.
 - **Conservación legal** mínima 10 años de XMLs firmados, respuestas DGII, TrackIds, status logs.
 - **Inmutabilidad** de los datos de cada e-CF emitido (no editable post-envío).
 - **Trazabilidad** completa desde proforma/factura origen → XML → firma → envío → respuesta.
-

15. Requisitos de seguridad

- **Cifrado at-rest:** Supabase encrypted storage + AES-256-GCM para blobs sensibles (.p12, password).
 - **Cifrado in-transit:** HTTPS obligatorio; HSTS habilitado.
 - **RLS:** todas las tablas con `business_id` tienen RLS enable + policy `business_id = auth_business_id()`.
 - **Mínimo privilegio:** roles del SaaS con permisos limitados; `service_role` solo para Edge Functions.
 - **Secrets management:** env vars sensibles `--sensitive` en Vercel; rotación documentada.
 - **OWASP Top 10:** input validation, output encoding, auth/session, CSRF tokens, secure cookies.
 - **Pentest:** anual con tester externo.
 - **Vulnerabilidad dependencias:** `npm audit` automático en CI; parches dentro de SLA.
-

16. Requisitos de auditoría

Eventos auditables (mínimo):

- Auth events: login, logout, password change, MFA enable/disable.
- DGII config: cert upload/revoke, settings update, sequence load.
- e-CF lifecycle: generate, sign, send, status check, accept, reject.
- Pre-Fase G: representative_attestation save, declaration accept/revoke.
- Ambiente: switch testecf → certecf → ecf (requiere autorización extra).
- Admin: rol change, permiso assign, business suspend.

Cada audit_log tiene: `business_id` (RLS), `user_id` (o NULL system), `user_name`, `action`, `entity`, `entity_id`, `metadata jsonb`, `ip_address`, `created_at`.

Retención: 10+ años. Backup inmutable mensual.

Acceso: `support` solo lectura; `auditor` lectura + exportación; `admin` ídem.

17. Requisitos multi-tenant

- **business_id NOT NULL** en todas las tablas del módulo (excepto root `businesses` y globales).
 - **RLS por business_id** en todas las tablas — sin excepciones.
 - **JWT claims** llevan `business_id`, `role`, `branch_id` para resolver tenant context.
 - **Helper auth_business_id()** lee claim en orden: root → app_metadata → user_metadata.
 - **Storage por business:** paths siempre prefijo `<business_id>/`.
 - **Tests cross-tenant:** smoke con 2 seed users; ninguno ve datos del otro.
 - **Billing separado** por business; sin agregaciones cross-tenant en UI.
 - **Suspension:** business `status='suspended'` bloquea emisión real DGII pero conserva datos.
-

18. Flujos UX

18.1 Onboarding

```
Signup → Bienvenida → /dgii (overview) → /dgii/habilitacion (wizard)
    → paso 1 cert → paso 2 config → ... → paso 10 estado final
```

18.2 Emisión e-CF (cajero)

```
/pos → seleccionar productos → cliente → /caja/cierre → confirmar
    → emitir e-CF → /dgii/facturas/[id] (status submitted)
    → cron actualiza status → cliente recibe PDF + QR
```

18.3 Aprobación comercial (recibido)

```
/dgii/recibidos → ver e-CF de tercero → verificar firma → aceptar / rechazar
    → enviar respuesta firmada al emisor
```


18.4 Soporte

Cliente reporta → soporte abre /admin/auditoria filtrado por business
→ ve último envío, error DGII, audit log
→ consulta runbook → guía al cliente o escala

19. Estados del proceso

Cada business tiene un estado global computado dinámicamente:

Estado	Condiciones de entrada	Condiciones de salida	Acciones permitidas
No iniciado	business creado, sin nada configurado	cualquier paso del wizard avanza	Configurar pasos 1-9
En preparación	algún paso comenzado, no todos completos	todos los pasos clave completed	Continuar wizard
Certificado cargado	cert activo válido, config fiscal incompleta	config fiscal completed	Completar config
Pendiente configuración fiscal	cert activo, sin RNC/razón en settings	RNC + razón guardados	Completar config
Pendiente postulación	técnicamente listo, sin postulación DGII confirmada	postulación confirmada por contador	Gestionar trámite externo
Pendiente secuencias	postulación OK, sin rangos cargados	rango cargado por tipo	Cargar /dgii/secuencias
Listo para testecf	cert+config+representante+postulación+rango	enviar primer e-CF a testecf	Habilitar DGII_TESTECF_SEND_ENABLED
Enviado a testecf	al menos 1 envío con TrackId	DGII responde aceptado/rechazado	Esperar status
Aprobado testecf	tasa aceptación > X% en período	solicitar transición a certecf	Iniciar certificación
Rechazado testecf	rechazos sin resolver	resolver y reenviar	Diagnosticar errores
En certificación	DGII certifica formalmente	DGII emite acta de certificación	Esperar resolución DGII
Certificado por DGII	acta recibida	activar producción fiscal	Cambiar ambiente=ecf
Listo para producción fiscal	acta + checklist § 26 cumplido	autorización dueño + admin + contador	Activar
Producción fiscal activa	ambiente=ecf + dgii_enabled_real_send=true	suspensión o cancelación	Operación normal

20. Requisitos por pantalla

20.1 /dgii (overview)

- **Propósito:** dashboard DGII del business.
- **Datos:** estado global, próximo paso, métricas (#e-CFs hoy/mes, % aceptados, cert vence en X días).
- **Acciones:** ir al wizard, ir a facturas, ir a configuración.
- **Gates:** rol $\geq$ cashier para ver; $\geq$ admin para acciones de config.
- **Errores:** sin DGII configurado $\rightarrow$ CTA "Empezar habilitación".

20.2 /dgii/habilitacion

- **Propósito:** wizard SaaS 10 pasos.
- **Datos:** 3 banners (MOCK / SaaS isolation / Pendiente testecf), panel global estado, 10 cards de paso, panel pre-flight runner.
- **Acciones:** click expandir paso, marcar items, completar evidencia, ejecutar pre-flight dry-run.
- **Gates:** auth seed user; rol admin para cambios.
- **Errores:** cert no cargado $\rightarrow$ paso 1 amber; XSD no encontrado $\rightarrow$ toast.

20.3 /dgii/certificado

- **Propósito:** lifecycle del cert.
- **Datos:** estado actual (sin cert / cargado / válido / vencido / inválido), metadata pública (subject, issuer, vigencia, fingerprint).
- **Acciones:** subir .p12 + password, ejecutar prueba local, revocar.
- **Gates:** rol con `dgii:certificate:upload`.
- **Errores:** password incorrecta $\rightarrow$ mensaje "Verificá la password del cert (no la imprimimos)"; cert vencido $\rightarrow$ bloqueo.

20.4 /dgii/configuracion

- **Propósito:** capturar `dgii_settings` del business.
- **Datos:** form con RNC, razón social, nombre comercial, dirección, provincia/municipio (selectores DGII), correo, teléfono, ambiente.
- **Acciones:** guardar, cambiar ambiente (con confirmación).
- **Gates:** rol con `dgii:configure`.
- **Errores:** RNC inválido (no 9/11 dígitos) $\rightarrow$ inline; cambio a `ecf` requiere autorización extra.

20.5 /dgii/secuencias

- **Propósito:** cargar rangos e-NCF por tipo + ambiente.
- **Datos:** tabla con tipo, ambiente, range_start, range_end, next_number, status, expires_at.

- **Acciones:** agregar rango, editar, marcar exhausted.
- **Gates:** rol con `dgii:sequences:manage`.
- **Errores:** `range_start > range_end` → inline; overlap con rango existente → bloqueo.

20.6 `/dgii/certificacion`

- **Propósito:** panel de pre-certificación mock para los 4 tipos.
- **Datos:** botón por tipo (31/32/33/34) + última corrida + evidencia.
- **Acciones:** ejecutar prueba por tipo.
- **Gates:** rol con `dgii:certification:run_tests`.
- **Errores:** XSD rechazo → mostrar primeros 3 errores con línea.

20.7 `/dgii/reportes`

- **Propósito:** reportes fiscales.
- **Datos:** filtros (período, tipo, status), totales (#emitidos, total ITBIS, total montos).
- **Acciones:** exportar CSV.
- **Gates:** rol con `dgii:reports:view`.

20.8 `/dgii/facturas`

- **Propósito:** listado y detalle de e-CFs.
- **Datos:** lista paginada con eNCF, fecha, cliente, monto, status, ambiente. Detalle: XML, respuesta DGII, TrackId, status_logs.
- **Acciones:** ver detalle, descargar XML/PDF, crear NC.
- **Gates:** rol con visualización.

20.9 `/admin/permisos`

- **Propósito:** catálogo y asignación de permisos.
- **Datos:** lista de permisos DGII + asignación por rol.
- **Acciones:** asignar/quitar permiso a rol.
- **Gates:** rol admin.

20.10 `/pos`

- **Propósito:** punto de venta.
- **Datos:** productos, carrito, cliente.
- **Acciones:** emitir comprobante (que invoca el flow e-CF si DGII enabled).
- **Errores:** cert vencido en mid-venta → mensaje guiado.

20.11 /caja/cierre

- **Propósito:** cierre de caja diario.
 - **Datos:** ventas del día agrupadas.
 - **Acciones:** confirmar cierre.
 - **Errores:** envíos DGII pendientes → mostrar antes de cerrar.
-

21. Criterios de aceptación por pantalla

Cada pantalla del § 20 debe pasar:

- ☒ Carga < 1s p95 sin DGII en el path.
 - ☒ Mobile responsive.
 - ☒ Sin errores en consola (production build).
 - ☒ Tests unitarios para lógica + integration para flows clave.
 - ☒ Mensajes en español RD claros para usuario no-técnico.
 - ☒ Logs estructurados server-side; sin secretos.
-

22. Criterios de aceptación por fase

Cada estado del § 19 debe cumplir su transición sin atajos:

- ☒ Condiciones de entrada documentadas en evaluador.
 - ☒ Tests automatizados para cada transición.
 - ☒ UI refleja el estado en < 1s tras cambio.
 - ☒ Audit log de la transición.
-

23. Casos de error

Código	Caso	Mensaje usuario final	Acción soporte
ERR-CERT-01	Password cert incorrecta	"La contraseña del certificado es incorrecta. Probá de nuevo."	Cliente verifica con quien le entregó el cert
ERR-CERT-02	Cert vencido	"Tu certificado venció el DD-MM-YYYY. Renová con tu CA emisora."	Soporte ayuda con CA
ERR-CERT-03	Cert RNC ≠ business RNC	"El RNC del certificado no coincide con tu RNC registrado."	Contador valida designación
ERR-XSD-01	XML rechazado por XSD	"El comprobante no cumple el formato DGII. Revisá ."	Soporte mira detalle XSD
ERR-SIGN-01	Firma falla	"No pudimos firmar el comprobante. Volvé a intentar."	Devops revisa Edge Function
ERR-DGII-401	Token DGII inválido	"DGII no autorizó el envío. Verificá postulación."	Soporte + contador validan SIRTSS
ERR-DGII-403	Contribuyente no autorizado	"Tu RNC no está habilitado para este ambiente DGII."	Soporte + contador
ERR-DGII-429	Rate limit DGII	"Demasiados envíos. Reintentando en 5 min."	Sistema auto-retry
ERR-DGII-5XX	DGII caída	"DGII no responde. Tus comprobantes se enviarán cuando vuelva."	DevOps alerta
ERR-SEQ-01	Secuencia agotada	"Se agotó el rango eNCF tipo X. Solicitá nuevo rango a DGII."	Soporte + contador
ERR-SEQ-02	Secuencia no autorizada	"El rango no está activo en DGII para este ambiente."	Contador verifica
ERR-AUTH-01	Sin permisos	"Tu rol no permite esta acción. Pedí a tu admin."	Soporte revisa permisos
ERR-RLS-01	Cross-tenant attempt	(no se muestra; bloqueado por DB)	Alarma de seguridad
ERR-STORAGE-01	Storage Supabase down	"Servicio temporal no disponible. Reintentá en 1 min."	DevOps
ERR-FLAG-01	DGII_TESTECF_SEND_ENABLED off	"Envío real bloqueado. Contactá a tu admin."	Soporte indica activación

24. Mensajes para usuario final

- "Tu certificado venció. Renová con tu autoridad certificadora para seguir emitiendo."
 - "Tu rango de comprobantes está al 90%. Solicitá uno nuevo a DGII."
 - "DGII está temporalmente no disponible. Tus comprobantes se enviarán automáticamente cuando vuelva."
 - "Listo. Tu comprobante fue aceptado por DGII (TrackId: XXX)."
 - "DGII rechazó tu comprobante: . Hablá con tu contador para resolverlo."
 - "Estás en modo prueba (testecf). Estos comprobantes no son fiscales todavía."
-

25. Mensajes para contador

- "El RNC del cert (NNN) no coincide con dgii_settings.rncEmisor (MMM). Verificá la designación de Usuario Administrador e-CF."
 - "Postulación testecf pendiente para RNC XXX. Iniciá el trámite en SIRTSS antes del envío."
 - "Rango eNCF tipo 31 testecf no cargado. Solicitá el rango a DGII y cargalo en /dgii/secuencias."
 - "Las 9 evidencias del paso 8 están confirmadas. Archivá el acta firmada con tu evidencia."
-

26. Mensajes para soporte

- `error.code=DGII_401 cause=token_expired retry_advice=re-execute_semilla`
 - `error.code=XSD_REJECTED cause=missing_FechaVencimientoSecuencia retry_advice=check_settings`
 - `error.code=SEQUENCE_EXHAUSTED business_id=XXX tipo=31 next_number=99999 range_end=99999`
-

27. Requisitos de documentación

- **Runbook operativo** (`docs/dgii/runbook-fase-f-g-h.md`) actualizado por fase.
 - **QA Checklist manual** (`docs/dgii/qa-saas-pre-fase-g.md`) para validación pre-go-live.
 - **Plan Maestro** (este paquete).
 - **Manual del contador** (subset del plan + glosario fiscal).
 - **Training material:** videos cortos por paso del wizard.
 - **API docs:** OpenAPI/Swagger interno para los route handlers.
-

28. Riesgos y mitigaciones

(Ver § 24 del Plan Maestro — riesgos R-01 a R-15 con probabilidad/impacto/mitigación.)

29. Dependencias

- **DGII** (`ecf.dgii.gov.do`) — externa, sin SLA público.
 - **Supabase** (DB + Auth + Storage + Edge) — 99.9% SLA.
 - **Vercel** (hosting + serverless) — 99.99% SLA.
 - **Entidad certificadora** (CA INDOTEL: Viafirma, Avansi, Certi-Empresa, GoDaddy DR) — cert se compra offline.
 - **Contador / asesor fiscal del cliente** — validaciones externas.
-

30. Fuera de alcance

- ERP completo (inventario complejo, RRHH, nómina).
 - Contabilidad doble entrada.
 - Asesoría fiscal personalizada.
 - NCF físicos legados (pre-2019).
 - Recuperación de cert perdido.
 - Automatizar postulación DGII (trámite externo del cliente).
 - Validación de identidad del titular (la CA emisora lo hace).
 - Sustituir al contador (auxiliar).
-

31. Criterios para go-live

Antes de activar producción fiscal real para un cliente:

- ☐ Cert vigente > 90 días.
- ☐ Postulación DGII aprobada formalmente.
- ☐ Rangos e-NCF de producción asignados por DGII para cada tipo.
- ☐ Margen suficiente: `next_number ≤ range_end - 100` .
- ☐ Acta de habilitación DGII recibida y archivada.
- ☐ Contador firmó declaración jurada interna.
- ☐ 7-30 días de envíos testecf con tasa aceptación > 95%.
- ☐ Backup Supabase del día (restorable).
- ☐ Cron polling status Fase H activo y monitoreado.
- ☐ Alarmas configuradas (rechazo, timeout, secuencia, cert vencimiento).
- ☐ Plan de rollback documentado y simulado.
- ☐ Soporte capacitado para errores comunes.
- ☐ Runbook actualizado para "DGII caída".
- ☐ Política de retención (10 años) implementada.

- ☐ Comunicación a clientes finales: emisión activa.

Solo con los 15 verdes: `ambiente='ecf'` + `dgii_enabled_real_send=true` + `DGII_TESTECF_SEND_ENABLED` permanece `false` (eso es solo para testecf).

Fin del PRD. Ver `agentes-y-skills-saas-dgii.md` para roles operativos.

Capítulo 3 — Agentes y Skills

Agentes y Skills — SaaS DGII e-CF para RD

Versión: 1.0 · **Fecha:** 2026-05-21 Catálogo de 25 agentes especializados, skills necesarios y prompts reutilizables para implementar y operar un módulo SaaS de facturación electrónica DGII desde cero.

1. Introducción

Este documento define los **agentes** que intervienen en el ciclo de vida del módulo SaaS DGII e-CF. Un "agente" puede ser:

- Una **persona** en un rol especializado (arquitecto, contador, ingeniero backend, etc.).
- Un **equipo pequeño** que comparte responsabilidad.
- Un **agente IA** con prompt dedicado (subagent en un sistema multi-agente).

La asignación humano vs IA depende de cada proyecto. La estructura de responsabilidades, skills, prompts y criterios de éxito es la misma. En equipos pequeños, una persona puede asumir múltiples agentes; en equipos grandes, un agente puede tener un team.

Convención de plantilla por agente:

```
## N. Agente <Nombre>
**Objetivo** – una línea
**Responsabilidad** – bullets concretos
**Skills necesarios** – lista
**Entradas / Salidas** – qué recibe y produce
**Herramientas** – tools, libs, servicios
**Prompt sugerido (IA)** – blockquote reusable
**Riesgos que vigila** – lista
**Criterios de éxito** – cómo se valida
**Checklist de trabajo** – tareas [ ]
```

2. Los 25 agentes

1. Agente Orquestador del Proyecto

Objetivo: Mantener la coherencia global del proyecto y asegurar que cada fase avance sin saltarse gates.

Responsabilidad:

- Aprobar transición entre fases del roadmap.
- Coordinar entre agentes técnicos, fiscales y operativos.

- Mantener PROJECT_MEMORY actualizado.
- Confirmar autorizaciones explícitas para Fase G/H/producción.
- Bloquear avances cuando faltan requisitos externos (postulación, rango).

Skills: Gestión de proyecto, comunicación con stakeholders no-técnicos, lectura de runbooks, manejo de killswitches, decisiones bajo incertidumbre.

Entradas: Estado actual del repo + criterios de salida de la fase actual. **Salidas:** Decisiones de avance documentadas en PROJECT_MEMORY + audit log.

Herramientas: Git, repo de docs, comunicación (slack/email), audit_logs UI.

Prompt sugerido (IA):

Sos el orquestador de un proyecto SaaS de facturación electrónica DGII para RD. Antes de aprobar el avance a una nueva fase, verificá: (1) la fase actual está completada según `roadmap-fases-saas-dgii.md` ; (2) los criterios de salida están todos verdes; (3) las autorizaciones externas (contador, dueño, DGII) están confirmadas; (4) los killswitches relevantes siguen en su default seguro. Si algún ítem falla, NO autorices; explicá qué falta. Nunca asumas Fase G ni producción fiscal sin instrucción explícita.

Riesgos que vigila: Avance prematuro a Fase G, producción fiscal por error, pérdida de trazabilidad entre fases.

Criterios de éxito: Ninguna fase se completa sin que las anteriores estén verdes; cero rollbacks por mala coordinación.

Checklist de trabajo:

- ☐ PROJECT_MEMORY actualizado por sesión.
- ☐ Audit logs revisados antes de aprobar avance.
- ☐ Killswitches default verificados.
- ☐ Stakeholders notificados de cada cambio de fase.
- ☐ Plan de rollback presente.

2. Agente Producto / PRD

Objetivo: Mantener el PRD vivo y los criterios de aceptación claros para cada pantalla y estado.

Responsabilidad:

- Actualizar PRD con nuevos requisitos.
- Definir criterios de aceptación por pantalla.
- Validar mensajes para usuario final / contador / soporte.
- Priorizar backlog en función de bloqueos del cliente.

Skills: Product management, UX writing, comunicación con usuarios, priorización.

Entradas: Feedback de clientes, métricas, requisitos DGII. **Salidas:** PRD actualizado, copy decks, criterios de aceptación.

Herramientas: Documentación markdown, mockups (Figma o similar).

Prompt sugerido (IA):

Sos el product owner del módulo SaaS DGII e-CF. Tu output principal es PRD vivo, criterios de aceptación por pantalla y mensajes claros para tres audiencias: dueño no-técnico, contador, soporte. Cuando agregues un requisito, asegurate de: (1) identificar la pantalla afectada; (2) definir criterio binario de aceptación; (3) escribir mensaje en español RD sin jerga técnica para el usuario final; (4) escribir mensaje técnico-fiscal para contador; (5) escribir diagnóstico para soporte.

Riesgos: Requisitos ambiguos, mensajes técnicos al usuario, falta de criterios binarios.

Criterios de éxito: PRD pasa revisión semestral; mensajes son comprensibles sin asistencia.

Checklist:

- ☐ PRD revisado.
- ☐ Mensajes validados con soporte.
- ☐ Mockups vinculados a requisitos.

3. Agente Compliance DGII

Objetivo: Asegurar que el módulo cumple Norma DGII 06-2018 y actualizaciones.

Responsabilidad:

- Monitorear cambios normativos DGII.
- Validar que el builder XSD-compliant respeta el XSD vigente.
- Auditar la separación testecf / certecf / ecf.
- Documentar requisitos legales en cada release.

Skills: Lectura de normativa DGII, XML/XSD, fiscalidad RD.

Entradas: Norma 06-2018, comunicados DGII, matriz de requisitos. **Salidas:** Reporte de compliance trimestral, alertas de cambios normativos.

Herramientas: Portal DGII, doc oficial, repo de XSDs.

Prompt sugerido:

Sos el agente de Compliance DGII. Tu tarea es leer las normas y comunicados DGII y mapearlos a requisitos del sistema. Cuando DGII publique un nuevo XSD, validá que el builder + validator del SaaS lo soportan. Cuando cambien las URLs de un ambiente, actualizá los defaults en `testecf-client.ts`. Tus salidas deben ser cambios concretos al repo, no notas genéricas.

Riesgos: Cambio normativo sin reflejo en código; XSD desactualizado.

Checklist:

- ☐ Norma vigente leída y mapeada.
 - ☐ XSDs en docs/dgii/xsd/ sincronizados con DGII.
 - ☐ URLs DGII actualizadas.
-

4. Agente Contable / Fiscal

Objetivo: Validar configuración fiscal del cliente y manejar la relación con DGII.

Responsabilidad:

- Confirmar postulación testecf/certecf del RNC.
- Validar rangos e-NCF asignados.
- Verificar designación de Usuario Administrador e-CF.
- Acompañar al cliente en errores DGII de origen fiscal.

Skills: Contabilidad RD, fiscalidad e-CF, portal SIRTSS DGII.

Entradas: Datos del contribuyente, acta DGII, evidencias. **Salidas:** Confirmación firmada de las 4 validaciones externas, dictamen fiscal.

Herramientas: SIRTSS DGII, calculadora ITBIS, normativa.

Prompt sugerido:

Sos el contador del contribuyente. Antes de autorizar Fase G real: (1) confirmá que la postulación testecf está aprobada en SIRTSS; (2) que el rango e-NCF está asignado y vigente; (3) que el titular del cert es Usuario Administrador e-CF designado para el RNC; (4) que el RNC del cert coincide con el contribuyente. Firmá las 4 evidencias en el paso 8 del wizard.

Riesgos: Avanzar sin postulación; rango incorrecto; designación expirada.

Checklist:

- ☐ Postulación testecf verificada.
 - ☐ Rango e-NCF cargado.
 - ☐ Acta de designación archivada.
 - ☐ Declaración formal firmada.
-

5. Agente Arquitectura SaaS

Objetivo: Diseñar la arquitectura multi-tenant + integraciones DGII.

Responsabilidad:

- Decidir stack y patrones.
- Diseñar separación de capas (UI / API / services / persistencia).

- Documentar decisiones técnicas con justificación.
- Revisar PRs estructurales.

Skills: Arquitectura web moderna, multi-tenant, edge computing, criptografía aplicada.

Entradas: Requisitos PRD, restricciones (Vercel, Supabase). **Salidas:** ADRs (Architecture Decision Records), diagramas, revisión de PRs.

Herramientas: Diagramas ASCII / Mermaid, docs.

Prompt sugerido:

Sos el arquitecto del SaaS. Antes de aprobar un cambio estructural: ¿respetas el aislamiento por business_id? ¿Sigues el patrón service puro + route handler? ¿Mantiene los killswitches? ¿No introduce un nuevo nivel de exposición de cert? Documentá cada decisión como ADR en docs/architecture/ .

Riesgos: Acoplamiento entre tenants; bypass de RLS; secretos expuestos.

Checklist:

- ☐ ADR escrito.
- ☐ Diagramas actualizados.
- ☐ PR revisado con foco arquitectónico.

6. Agente Supabase / Postgres / RLS

Objetivo: Diseñar y operar la base de datos multi-tenant con RLS.

Responsabilidad:

- Escribir migraciones idempotentes.
- Definir políticas RLS por tabla.
- Optimizar índices y queries.
- Auditar performance.

Skills: PostgreSQL avanzado, Supabase SDK, RLS, JSONB, funciones SQL, índices.

Entradas: Modelo de datos del PRD. **Salidas:** Migraciones .sql aplicadas, scripts de verificación RLS.

Herramientas: Supabase Studio, psql, MCP Supabase.

Prompt sugerido:

Sos el DBA del SaaS. Cada migración que escribas debe ser: (1) idempotente (CREATE TABLE IF NOT EXISTS , CREATE POLICY con guard); (2) aditiva (sin DROP que pierda data); (3) RLS-aware (toda tabla con business_id tiene policy); (4) testeable (smoke test cross-tenant disponible). Después de aplicar, corré un test que verifique que un user del business A NO puede ver datos del business B.

Riesgos: Migración destructiva, RLS faltante, índice mal puesto.

Checklist:

- ☐ Migración idempotente.
 - ☐ RLS policy definida.
 - ☐ Smoke cross-tenant.
 - ☐ Backup pre-migración.
-

7. Agente Seguridad / Certificados

Objetivo: Custodiar el ciclo de vida del certificado digital del contribuyente.

Responsabilidad:

- Diseñar cifrado AES-256-GCM del blob + password.
- Definir rotación de keys.
- Auditar accesos al cert.
- Validar que el cert nunca se expone al cliente.

Skills: Criptografía aplicada, PKCS#12, KMS/Vault, AES-GCM.

Entradas: Requisitos de seguridad + cert del contribuyente. **Salidas:** Servicios `certificate-storage.ts`, `cert-cipher.ts` testeados.

Herramientas: node-forge, node:crypto, Supabase Storage.

Prompt sugerido:

Sos el guardián del certificado. Tu mantra: "el `.p12` nunca llega al cliente, la password nunca se imprime, la key de cifrado nunca se commitea". Cualquier código que toque cert pasa por tu revisión. Si ves un `console.log` cerca de material sensible, lo eliminás. Si ves una key sin `--sensitive` en Vercel, lo arreglás.

Riesgos: Leak de cert, leak de password, leak de key.

Checklist:

- ☐ AES-256-GCM verificado.
 - ☐ Sin logs de material sensible.
 - ☐ Key de cifrado backed up.
 - ☐ Tests anti-leak en suite.
-

8. Agente Backend e-CF

Objetivo: Implementar y mantener los services puros del módulo.

Responsabilidad:

- Builder XML XSD-compliant.

- Signer XMLDSig.
- Validator XSD.
- Cliente DGII testecf (preflight + execute).
- Tests unitarios + integration.

Skills: TypeScript estricto, XML, XMLDSig, testing.

Entradas: XSDs DGII, tipos del PRD. **Salidas:** Modules en `apps/web/src/server/services/dgii/`.

Herramientas: node-forge, xml-crypto, xmllint-wasm, xmlbuilder2, vitest.

Prompt sugerido:

Sos el responsable del backend e-CF. Cada PR debe incluir: (1) test del happy path; (2) test de un edge case; (3) test que valida no-leak de secretos; (4) typecheck verde; (5) ningún `import "server-only"` faltante. El builder respeta el orden EXACTO del XSD. El signer usa `isEmptyUri:true`. El validator usa `patchOfficialDgiiXsd` para el tipo del XSD 31.

Riesgos: XML que falla XSD, firma incorrecta, leak en tests.

Checklist:

- ☐ Tests verdes.
- ☐ XSD oficial valida output del builder.
- ☐ Firma se verifica con clave pública.

9. Agente XML / XSD

Objetivo: Mantener el contrato entre el XML generado y el XSD oficial DGII.

Responsabilidad:

- Importar y normalizar XSDs oficiales.
- Documentar tipos / fixes de DGII.
- Validar que el builder respeta el orden XSD.
- Diagnosticar rechazos por schema.

Skills: XML Schema, namespaces, canonicalization, libxml2.

Entradas: XSDs publicados por DGII. **Salidas:** Tests de `validateEcfXml` por tipo + parches documentados.

Herramientas: xmllint-wasm, libxml docs.

Prompt sugerido:

Sos el experto XSD. Cuando DGII publique un nuevo XSD, lo descargás, lo poneés en `docs/dgii/xsd/`, escribís tests para él, y documentás cualquier tipo / inconsistencia conocida en `validator.ts` con un

parche server-side que normalice antes de validar. Nunca modifiques el archivo XSD en disco — siempre parche en memoria.

Riesgos: XSD desactualizado, typo no documentado, falsos positivos.

Checklist:

- ☐ XSDs sincronizados.
- ☐ Tests por tipo (31/32/33/34).
- ☐ Parches documentados.

10. Agente Firma Digital / XMLDSig

Objetivo: Implementar XMLDSig enveloped según DGII.

Responsabilidad:

- Firmar XML con `xml-crypto`.
- Verificar firma con clave pública.
- Manejar canonicalization correcta.
- Asegurar `Reference URI=""` + `isEmptyUri:true`.

Skills: XMLDSig, c14n, RSA, X.509.

Entradas: XML sin firmar + cert PEM + key PEM. **Salidas:** XML firmado verificable.

Herramientas: xml-crypto, @xmldom/xmldom, node:crypto.

Prompt sugerido:

Sos el firmador. Tu output: un XML con `<Signature>` enveloped al final de `<ECF>`, con `SignatureMethod=rsa-sha256`, `DigestMethod=sha256`, `CanonicalizationMethod=c14n-20010315`, `Reference URI=""` (`isEmptyUri:true`). Nunca dejes el cert PEM con headers; siempre limpiá a base64 puro dentro de `<X509Certificate>`. Tras firmar, ejecutá `verifyEcfSignature` para confirmar consistencia.

Riesgos: Firma incorrecta, XSD rechaza por Id en ECF, c14n equivocada.

Checklist:

- ☐ Firma verifica.
- ☐ XSD oficial pasa.
- ☐ Sin Id agregado a ECF.

11. Agente Integración DGII testecf

Objetivo: Cablear el flow auth + recepción contra DGII testecf.

Responsabilidad:

- Implementar Semilla → ValidarSemilla → bearer token.
- Implementar POST multipart `/repcion/api/ecf`.
- Persistir request/response en `dgii_submissions`.
- Manejar errores 4xx/5xx con retry.

Skills: HTTP avanzado, multipart, OAuth-like flows, error handling.

Entradas: XML firmado + cert + endpoints. **Salidas:** TrackId persistido + dgii_submissions audit.

Herramientas: fetch, FormData, AbortController.

Prompt sugerido:

Sos el cliente DGII testecf. NUNCA hagás fetch en una versión que se autorice como "dry-run". Cuando se autorice "live", respetá: (1) cache del token mientras no expire; (2) reintento con backoff en 5xx; (3) timeout de 30s; (4) zero logs del bearer token; (5) persistir request_body y response_body en Storage cifrado, no en logs. Todo cambio aquí pasa por revisión de seguridad y compliance.

Riesgos: Endpoint prod por error, token leak, retry infinito.

Checklist:

- ☐ testecf URL hardcoded como default.
- ☐ Override env validado.
- ☐ Backoff implementado.
- ☐ Test con fetch mockeado.

12. Agente TrackId / Status

Objetivo: Implementar polling de status DGII (Fase H).

Responsabilidad:

- Cron Edge Function para invoices con `status IN ('submitted','in_process')`.
- Consultar `/consultatrackid/api/TrackIds/Estado`.
- Mapear códigos DGII a `electronic_invoices.status`.
- INSERT en `dgii_status_logs`.

Skills: Edge Functions, cron, rate limit, mapeo de códigos.

Entradas: track_ids pendientes. **Salidas:** status actualizado.

Herramientas: Vercel Cron, fetch, dgii_status_logs.

Prompt sugerido:

Sos el responsable de Fase H. Tu cron corre cada N minutos. Por cada track_id pendiente: consultá DGII con rate limit (max 60/min por business); mapeá la respuesta a uno de los estados oficiales (Aceptado,

Aceptado Condicional, Rechazado, En Proceso); INSERT log; actualizá `electronic_invoices` . Si DGII responde 429, degradá a polling cada 30 min hasta que vuelva.

Riesgos: Polling abusivo, leak de tokens en cron logs, mapeo incorrecto.

Checklist:

- ☐ Rate limit aplicado.
- ☐ Logs sin tokens.
- ☐ Tests de mapeo.

13. Agente Frontend / UX SaaS

Objetivo: Construir UI clara, accesible, multi-rol, en español RD.

Responsabilidad:

- Wizard /dgii/habilitacion con UX no-técnica.
- Formularios con validación inline.
- Mensajes accionables.
- Mobile responsive.

Skills: React (RSC + client), Tailwind, accesibilidad, UX writing español.

Entradas: PRD, mockups, copy decks. **Salidas:** Componentes en `apps/web/src/app/(app)/dgii/*` .

Herramientas: Next.js App Router, Tailwind, design system del SaaS.

Prompt sugerido:

Sos el frontend SaaS. Cada pantalla debe ser comprensible por un dueño de negocio sin conocimientos técnicos. Mensajes en español RD, sin jerga; CTAs verbo + objeto ("Subir certificado", "Verificar pre-flight"). Sin emojis. Estados claros con badges. Botones críticos (envío real) DISABLED por default con tooltip que explica.

Riesgos: Copy técnico, falta de feedback, UX confusa.

Checklist:

- ☐ Copy revisado por contador.
- ☐ Accesibilidad básica (focus, aria).
- ☐ Mobile responsive.

14. Agente POS / Caja

Objetivo: Integrar la emisión e-CF al flow del punto de venta.

Responsabilidad:

- Cobrar y emitir e-CF en < 30s.
- Manejar DGII caída con cola.
- Imprimir representación con QR.

Skills: UX cajero, impresoras térmicas, manejo de errores DGII en tiempo real.

Entradas: Productos + cliente. **Salidas:** e-CF emitido + impresión.

Prompt sugerido:

Sos el responsable del POS. El cajero NO sabe DGII. Si DGII está caída, dejá que cobre y encolá; mostrá "Comprobante en proceso". Si el cert venció, BLOQUEÁ con mensaje claro y CTA a contador. Si el rango está al 90%, banner amber permanente.

Riesgos: Caja bloqueada por DGII, comprobante perdido en cola.

Checklist:

- ☐ Cola funcional.
- ☐ Impresión 80mm OK.
- ☐ Mensajes claros.

15. Agente QA Automatizado

Objetivo: Mantener suite automática que prohíbe regresiones.

Responsabilidad:

- Tests unitarios builder + signer + validator.
- Tests cross-tenant RLS.
- Tests anti-leak de secretos.
- Tests con fetch mockeado (zero red en CI).

Skills: Vitest, spies, mocks, fakes.

Entradas: Código + PRD. **Salidas:** Suite vitest con cobertura > 90% services.

Herramientas: Vitest, vi.spyOn, vi.mock.

Prompt sugerido:

Sos el QA automatizado. Cada PR sin tests va de regreso. Spy global de `fetch` instalado en `beforeAll` que TIRA si alguien lo invoca — garantía dura de "ningún test golpea DGII". Tests de RLS con seed users distintos. Tests que validan `JSON.stringify(evidence)` no contiene "BEGIN PRIVATE KEY".

Riesgos: Test que toca red real, cobertura caída, regresión silenciosa.

Checklist:

- ☐ vitest verde 100%.

- ☐ fetch spy verifica zero invocaciones.
 - ☐ Cobertura > 90% en services.
-

16. Agente QA Manual Cliente

Objetivo: Validar el flow end-to-end como cliente real en Preview.

Responsabilidad:

- Seguir checklist `qa-saas-pre-fase-g.md`.
- Reportar bugs UX y de copy.
- Validar mensajes para no-técnicos.

Skills: QA manual, atención al detalle, paciencia.

Entradas: Preview URL + checklist QA. **Salidas:** Reporte 14/14 con evidencia screenshot.

Prompt sugerido:

Sos el QA manual. Tu test es un cliente nuevo, no-técnico, intentando habilitar su negocio. Si algo te confunde, lo reportás. Si un botón no se entiende, falla el QA. Si un mensaje tiene jerga, falla.

Riesgos: Falsos positivos por familiaridad.

Checklist:

- ☐ 14/14 criterios verdes.
 - ☐ Screenshots adjuntos.
 - ☐ Issues documentados.
-

17. Agente DevOps / Vercel

Objetivo: Mantener pipelines de deploy y env vars sin contaminar producción.

Responsabilidad:

- Preview deploys por PR.
- Env vars separadas (Preview vs Production).
- `outputFileTracingIncludes` para XSDs.
- Rollback < 1 min.

Skills: Vercel, env management, CI/CD.

Entradas: Branch + commits. **Salidas:** Preview URL + Production deploy autorizado.

Prompt sugerido:

Sos el DevOps del SaaS. Tu regla #1: producción Vercel está VACÍA hasta que el cliente tenga su certificación DGII completa. Cada env var en Production se revisa con compliance. `vercel deploy --prod` requiere checklist go-live + autorización del dueño SaaS.

Riesgos: Env var con secret en Production, deploy prod accidental.

Checklist:

- ☐ Preview por PR.
- ☐ Production env minimal.
- ☐ Rollback testado.

18. Agente Documentación / Runbooks

Objetivo: Mantener docs operativos vivos.

Responsabilidad:

- Runbook por fase.
- Onboarding del cliente.
- Glosario.
- PROJECT_MEMORY al cierre de cada sesión.

Skills: Escritura técnica clara, organización, español RD.

Entradas: Commits + sesiones de trabajo. **Salidas:** Markdown en `docs/dgii/`.

Prompt sugerido:

Sos el documentador. Tras cada cambio significativo, actualizá: (1) runbook de la fase afectada; (2) PROJECT_MEMORY con sesión + lecciones; (3) docs/estado-actual.md con el snapshot. Tu objetivo: que un dev nuevo pueda onboardear leyendo solo los docs en orden.

Riesgos: Docs desactualizados, knowledge en cabeza de una persona.

Checklist:

- ☐ Runbook al día.
- ☐ PROJECT_MEMORY actualizado.
- ☐ Glosario revisado.

19. Agente Auditoría / Logs

Objetivo: Garantizar trazabilidad fiscal completa.

Responsabilidad:

- audit_logs INSERT policy aplicada.
- Retención 10 años.
- Exportación por business.

Skills: Postgres, RLS, JSONB, backups, compliance.

Entradas: Eventos del sistema. **Salidas:** audit_logs íntegro + exportaciones.

Prompt sugerido:

Sos el auditor del sistema. Cada acción fiscal del módulo DEBE generar un audit_log. Si encontrás un flow sin audit (ej. cambio de ambiente sin log), agregalo. Validá que la INSERT policy bloquea suplantación (`user_id = auth.uid()`).

Riesgos: Eventos sin log, retención incumplida, RLS bypass.

Checklist:

- ☐ audit_logs sin huecos.
- ☐ Retención 10 años configurada.
- ☐ Export funcional.

20. Agente Soporte / Onboarding Cliente

Objetivo: Acompañar al cliente del SaaS de signup a producción fiscal.

Responsabilidad:

- Guiar wizard.
- Resolver dudas comunes.
- Escalation a técnico / compliance.
- Capacitación en errores DGII.

Skills: Comunicación, empatía, conocimiento del producto, runbook.

Entradas: Tickets + chat clientes. **Salidas:** Tickets resueltos + lecciones para FAQ.

Prompt sugerido:

Sos soporte SaaS DGII. Acompañás al cliente paso a paso. NUNCA das asesoría fiscal (eso lo hace el contador). Si el cliente pregunta "¿debería emitir esto?", redirigís a su contador. Si el problema es técnico, escalás. Documentás cada caso recurrente en la FAQ.

Riesgos: Asesoría fiscal indebida, sin escalation matrix.

Checklist:

- ☐ Tickets respondidos < SLA.
- ☐ FAQ actualizada.

- ☐ Escalation seguida.

21. Agente Seguridad de Producción

Objetivo: Asegurar que producción fiscal opera segura.

Responsabilidad:

- Audit de env vars Production.
- Pentest anual.
- Monitoreo de leaks (Sentry / similar).
- Revisión de accesos.

Skills: OWASP, secrets management, observability.

Prompt sugerido:

Sos seguridad de producción. Tu rutina: revisar `vercel env ls production` semanal; verificar que ninguna var sensible está unmasked; correr `npm audit` mensual; revisar Sentry por leaks.

Riesgos: Env con secret, dep con vulnerabilidad, leak en error report.

Checklist:

- ☐ env Production sano.
- ☐ npm audit verde.
- ☐ Sentry sin PII.

22. Agente Migración entre SaaS

Objetivo: Replicar el módulo a otros SaaS RD.

Responsabilidad:

- Empaquetar services + migraciones.
- Adaptar UI al sistema destino.
- Acompañar el clonado.

Skills: Conocimiento profundo del módulo, refactor, comunicación.

Prompt sugerido:

Sos el agente de migración. Cuando un nuevo SaaS quiera implementar e-CF DGII: copía `apps/web/src/server/services/dgii/*` + `supabase/migrations/000[1-7]*.sql` + `docs/dgii/xsd/`. Adaptá el wizard al design system del destino. Reusá los tests. Documentá las adaptaciones.

Checklist:

- ☐ Paquete portable.
 - ☐ Tests pasan en destino.
 - ☐ Docs adaptadas.
-

23. Agente Observabilidad / Monitoreo

Objetivo: Dashboards y alarmas.

Responsabilidad:

- Métricas por business.
- Alarmas críticas configuradas.
- Dashboards de Grafana/Vercel/Supabase.

Skills: Observability, métricas, alarms.

Prompt sugerido:

Sos observabilidad. Asegurate de que: (1) cada Edge Function emite métricas; (2) errores DGII se loggean con `business_id` ; (3) dashboards muestran tasa aceptación + latencia + volumen por business; (4) alarmas pagan oncall cuando rechazo > 5%.

Checklist:

- ☐ Métricas activas.
 - ☐ Alarmas testeadas.
 - ☐ Dashboards públicos.
-

24. Agente Respuesta a Incidentes

Objetivo: Manejar incidentes (DGII caída, breach, data loss).

Responsabilidad:

- Runbook por tipo de incidente.
- Post-mortem.
- Comunicación a clientes.

Skills: Crisis management, comunicación, root cause analysis.

Prompt sugerido:

Sos respuesta a incidentes. Si DGII está caída > 30 min: notificá a todos los clientes, activá cola, prepará comunicación. Si hay sospecha de breach: aislá, preserve evidencia, llamá compliance y legal. Post-mortem en < 48h, blameless.

Checklist:

- ☐ Runbook incidentes.
 - ☐ Post-mortems archivados.
-

25. Agente Capacitación Cliente

Objetivo: Onboarding educativo al cliente final.

Responsabilidad:

- Videos cortos por paso.
- Webinars trimestrales.
- Material PDF para contadores.

Skills: Producción de contenido, didáctica.

Prompt sugerido:

Sos capacitación. Tu output: videos de < 3 min por paso del wizard; PDF resumen para contadores; webinar trimestral con casos reales (anonimizados). Sin jerga.

Checklist:

- ☐ Video por paso.
 - ☐ PDF contador.
 - ☐ Webinar agendado.
-

3. Catálogo de Skills

3.1 Técnicos

Skill	Nivel mínimo	Cómo se valida	Recursos
TypeScript estricto	Intermedio	Code review + typecheck verde	Handbook oficial
Next.js (App Router)	Intermedio	Builds + RSC + Server Actions	nextjs.org/docs
React (RSC/Client)	Intermedio	Componentes sin hydration errors	react.dev
Supabase (Auth/Storage/DB/RLS)	Avanzado	Migración funcional + smoke RLS	supabase.com/docs
PostgreSQL avanzado	Avanzado	Diseño schema + RLS + funciones	postgresql.org
SQL (DDL/DML)	Intermedio	Migración correcta	DDD libro
XML / namespaces	Intermedio	Builder XSD-compliant	W3C XML
XSD validation	Avanzado	xmllint contra docs/dgii/xsd/	W3C XSD
XMLDSig (enveloped, c14n)	Avanzado	Firma verifica + XSD acepta	W3C XMLDSig
PKCS#12 (node-forge)	Avanzado	Parseo .p12 + extracción cert/key	node-forge docs
Cifrado AES-256-GCM	Avanzado	Sealed JSON + verificación	NIST SP 800-38D
API REST + multipart	Intermedio	POST /repcion exitoso	fetch MDN
Vercel	Intermedio	Preview deploys + outputFileTracing	vercel.com/docs
Multi-tenant SaaS	Avanzado	RLS + tests cross-tenant	Patterns of EAA
Vitest (mocks/spies)	Intermedio	Suite verde + spy fetch	vitest.dev
Seguridad de secretos	Avanzado	npm audit + no leaks en repo	OWASP secrets
Auditoría / trazabilidad	Avanzado	audit_logs cubierto	SOX/PCI
Manejo errores DGII	Intermedio	Códigos mapeados + retry	Norma 06-2018

3.2 Soft

Skill	Nivel mínimo	Validación
UX writing español RD	Intermedio	Mensajes comprensibles
Documentación técnica clara	Intermedio	Onboarding < 2 días
Comunicación con contador	Intermedio	Sin confusión fiscal
Soporte SaaS empático	Intermedio	NPS > 50
Gestión de incidentes	Avanzado	MTTR < SLA
QA manual con checklists	Básico	14/14 verdes
Análisis de logs	Intermedio	Diagnóstico < 30 min

3.3 Dominio

Skill	Nivel mínimo	Validación
Contabilidad y fiscalidad RD	Intermedio	Diagnóstico fiscal correcto
Norma DGII 06-2018	Avanzado	Cumplimiento auditado
e-CF tipos 31/32/33/34/41-47	Intermedio	Builder soporta tipos correctos
Postulación testecf/certecf/ecf	Avanzado	Trámite SIRTSS completo
Usuario Admin e-CF	Avanzado	Designación válida en DGII
Rangos e-NCF	Avanzado	Rango cargado + validado
Representación impresa	Intermedio	PDF con todos los campos
Código de seguridad / QR	Intermedio	Payload calculado correcto

4. Prompts reutilizables transversales

4.1 Análisis pre-fase

Antes de avanzar a la Fase N del roadmap SaaS DGII, revisá: (1) fase N-1 está completa con sus criterios de salida; (2) todos los killswitches relevantes siguen en su default seguro; (3) las autorizaciones externas (contador, dueño, DGII) están confirmadas y documentadas; (4) los riesgos top-3 de la nueva fase tienen mitigación. Si algún ítem falla, NO autorices el avance y listá qué falta.

4.2 Revisión de seguridad

Auditá este código por leaks de secretos. Buscá: (a) `console.log` cerca de passwords/JWTs/private keys/cert PEM; (b) env vars sin `--sensitive` en Vercel; (c) `JSON.stringify` de objetos que contienen material sensible; (d) errores que incluyen `cause` con info sensible. Reportá hallazgos con archivo:línea y fix sugerido. No sugieras workarounds; arreglá la causa.

4.3 Validación XSD

Generá un e-CF tipo N con datos demo (RNC xxxxxxxxxx , razón social "Demo SRL", monto 100, ITBIS 18%). Pasá el resultado por `validateEcFXML` contra `docs/dgii/xsd/e-CF-N-v1.0.xsd` . Si falla, mostrá los primeros 3 errores con línea + diagnóstico. NO modifiques el XSD; si hay typo, parchealo en `patchOfficialDgiiXsd` .

4.4 Diagnóstico error DGII

Recibí esta respuesta de DGII: `{status: NNN, body: "..."} .` Identificá: (a) categoría (postulación / sequence / token / schema / rate-limit / outage); (b) acción del usuario; (c) acción del soporte; (d) si requiere escalation a contador o compliance. Devolvé un objeto `{ category, userMessage (ES RD), supportLog, escalation }` .

4.5 Onboarding cliente

Guiá al cliente paso a paso desde signup hasta primer e-CF emitido en testecf. NO uses jerga técnica. Validá cada paso preguntando al cliente que confirme antes de avanzar. Si el cliente está bloqueado por algo externo (postulación, rango), explicalé qué hacer y a quién contactar (contador / DGII portal). Tu objetivo: que cumpla wizard 10 pasos sin abrir ticket técnico.

4.6 Revisión de migración Supabase

Esta es una migración nueva: `[contenido SQL]` . Validá: (1) idempotente (`IF NOT EXISTS` , guards); (2) RLS aplicada si tiene `business_id` ; (3) índices sobre `business_id` como primera columna; (4) FKs con ON DELETE adecuado; (5) sin DROP destructivo. Sugerí mejoras si las hay.

4.7 Pre-flight Fase G

Generá pre-flight dry-run para business X tipo e-CF Y: resolvé cert + dgii_settings + XSD; build XML; valida XSD; firma; calcula URLs testecf; lista razones de bloqueo del envío real. NO toques DGII. Devolvé el shape

5. Matriz fase → agentes

Fase	Agentes principales	Agentes de revisión
0 Diagnóstico	Orquestador, Arquitectura	Producto, Compliance
1 Base SaaS	Arquitectura, Supabase, DevOps	Seguridad
2 Modelo DGII	Supabase, Compliance, Backend e-CF	Auditoría
3 UI mock	Frontend, Producto	UX
4 Builder XML	Backend e-CF, XML/XSD	Compliance
5 XSD validation	XML/XSD, Backend	QA Auto
6 Firma demo	Firma Digital, Backend	Seguridad
7 Supabase real	Supabase, Seguridad	RLS, Auditoría
8 Wizard SaaS	Frontend, Producto	Soporte
9 Cert real	Seguridad/Cert, Backend	Compliance
10 Validación local	Backend, Firma, QA Auto	Seguridad
11 Pre-Fase G checklist	Contable, Frontend	Soporte, Compliance
12 Dry-run Fase G	Backend, Integración testecf, QA Auto	Seguridad, Arquitectura
13 Postulación + sequences	Contable, Cliente	Compliance
14 Fase G real	Integración testecf, TrackId	Seguridad, Compliance
15 Fase H polling	TrackId, Observabilidad	DevOps
16 Certificación DGII	Contable, Compliance	Orquestador
17 Producción fiscal	Orquestador, Seguridad Prod	DevOps, Compliance
18 Operación	Soporte, Observabilidad, Incidentes	Auditoría
19 Reutilización	Migración entre SaaS	Arquitectura, Doc
20 Escalamiento	Arquitectura, Observabilidad	DevOps

6. Cómo replicarlo

Para clonar este modelo de agentes a otro SaaS:

1. **Inventario humano vs IA:** identificá qué roles tenés cubiertos por personas vs cuáles delegás a sub-agentes IA.
2. **Reusá prompts:** los prompts de § 4 son starters; ajustá nombres de archivos al stack del nuevo proyecto.
3. **Adaptá la matriz fase→agente:** si tu proyecto no tiene equipo separado de Frontend, fusioná con Producto.
4. **Mantené killswitches:** independiente del tamaño del equipo, los 6 killswitches del módulo (DGII_TESTECF_SEND_ENABLED, ambiente check, postulación, rango, confirmación manual, execute stub) son no-negociables.
5. **Reusá skills catalog:** sirve también para entrevistas técnicas de roles nuevos.
6. **Documentá tus desviaciones:** si decidís implementar diferente (ej. usás KMS en vez de env var para `DGII_CERT_ENCRYPTION_KEY`), documentá la razón.

Aplica a:

- Otros SaaS dominicanos: el flow es idéntico.
- SaaS para CFDI (México), DTE (Chile), CPE (Perú): la estructura de agentes/skills aplica, pero los XSDs, endpoints y normativa cambian. El módulo concreto se reescribe; el patrón de agentes y skills se reusa.

Fin del catálogo de Agentes y Skills. Ver `roadmap-fases-saas-dgii.md` para el detalle por fase.

Capítulo 4 — Roadmap por Fases

Roadmap por Fases — SaaS DGII e-CF para RD

Versión: 1.0 · **Fecha:** 2026-05-21 20 fases con objetivo, entregables, criterios de salida, bloqueos y qué NO hacer.

Tabla resumen de las 20 fases

#	Fase	Objetivo	Estado DermaLand	Bloqueos para empezar	Duración estimada
0	Diagnóstico	Auditar sistema existente	✅ Completo	—	2-5 días
1	Base SaaS multi-tenant	Tenancy + auth + roles	✅ Completo	Fase 0	1-2 semanas
2	Modelo DGII/e-CF	Schema DB DGII	✅ Completo	Fase 1	1 semana
3	UI mock/demo	Pantallas con datos fake	✅ Completo	Fase 2	1-2 semanas
4	Builder XML e-CF	XSD-compliant tipos 31/32/33/34	✅ Completo	Fase 2	1-2 semanas
5	XSD validation	xmllint-wasm contra XSDs oficiales	✅ Completo	Fase 4	3-5 días
6	Firma digital demo	XMLDSig con cert dummy	✅ Completo	Fase 4	1 semana
7	Supabase real + RLS	Migraciones 0001-0006 aplicadas	✅ Completo	Fase 1	1-2 semanas
8	Wizard habilitación SaaS	10 pasos con evidencia	✅ Completo	Fase 3, 7	2 semanas
9	Certificado real en Preview	Upload + cifrado .p12	✅ Completo	Fase 7	1 semana
10	Validación local certificado	runLocalCertTest + XSD	✅ Completo	Fase 9	3-5 días
11	Pre-Fase G checklist	9 ítems + declaración	✅ Completo	Fase 8, 10	1 semana
12	Dry-run Fase G	prepareTestecfSubmission sin HTTP	✅ Completo	Fase 11	1 semana
13	Postulación + sequences	Trámite DGII + rangos	⌚ Externo cliente	Cliente real	Variable
14	Fase G testecf real	POST real a testecf	🔒 Bloqueada	Fase 13 + autorización	2-3 semanas
15	Fase H TrackId/status	Polling DGII	🔒 Bloqueada	Fase 14	1-2 semanas
16	Certificación DGII	DGII certifica formalmente	🔒 Bloqueada	Fase 14-15 estables	4-8 semanas
17	Producción fiscal	ambiente=ecf	🔒 Bloqueada	Fase 16	1-2 semanas
18	Operación + soporte	Monitoreo + runbooks	🔒 Bloqueada	Fase 17	Continuo
19	Reutilización	Empaquetar para otros SaaS	🔒 Pendiente	Fase 18 estable	2-3 semanas

#	Fase	Objetivo	Estado DermaLand	Bloqueos para empezar	Duración estimada
20	Escalamiento	Multi-cliente robusto	 Pendiente	Fase 18 + N clientes	Continuo

Detalle por fase

Fase 0 — Diagnóstico del sistema existente

Objetivo: auditar el sistema actual del SaaS (si existe) para identificar qué se aprovecha y qué se construye.

Descripción: Antes de codear nada DGII, hay que entender qué tiene el SaaS: auth, multi-tenant, POS, productos, clientes, persistencia. Esto evita duplicación y permite integrar el módulo en arquitectura existente.

Qué se construye: Un informe de gaps + plan de integración + ADR de decisiones técnicas. No hay código aún.

Módulos involucrados: N/A (solo análisis). **Archivos típicos:** `docs/dgii/auditoria-sistema-existente.md`, `docs/dgii/plan-integracion-dgii.md`. **Tablas típicas:** N/A. **Variables:** N/A. **Agentes:** Orquestador, Arquitectura, Producto, Compliance. **Skills:** Lectura de código, comunicación con stakeholders, análisis arquitectónico. **Dependencias:** Acceso al repo y a los stakeholders. **Riesgos:** Asumir sin validar lo que tiene el sistema; faltar entrevistas a contador. **Validaciones:** Informe leído y firmado por arquitectura + producto. **Pruebas:** N/A. **Criterios de salida:** Informe completo; gaps documentados; ADR aprobado. **Qué NO hacer todavía:** Codear migración; instalar deps DGII. **Autorización:** Lead técnico. **Entregables:** Informe + ADR.

Checklist de cierre:

- ☐ Repo auditado.
- ☐ Stakeholders entrevistados.
- ☐ ADR firmado.
- ☐ Plan aprobado.

Fase 1 — Base SaaS multi-tenant

Objetivo: Tener `businesses`, `users`, auth, roles, RLS básico funcionando.

Descripción: El módulo DGII opera sobre un SaaS multi-tenant. Si no existe, hay que construirlo primero. Si existe, validar que tiene `business_id` por tenant y RLS.

Qué se construye: Migración base; `auth_business_id()` SQL function; helper `getSession()` que lee JWT.

Archivos típicos: `supabase/migrations/0001_phase1_core.sql`, `apps/web/src/server/auth/context.ts`.

Tablas: `businesses`, `branches`, `users`, `plans`. **Variables:** `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`. **Agentes:** Arquitectura, Supabase/RLS, DevOps, Seguridad. **Skills:** PostgreSQL, RLS, Supabase Auth, Next.js middleware. **Dependencias:** Fase 0. **Riesgos:** RLS faltante en una tabla; JWT no llega a server; cross-tenant leak. **Validaciones:** Smoke con 2 seed users (business

A y B); A no ve B. **Pruebas:** Test E2E de tenant isolation. **Criterios de salida:** RLS activa en todas las tablas; 2 seed users probados; auth_business_id() responde claims correctos. **Qué NO hacer todavía:** Tablas DGII (Fase 2); pantallas DGII. **Autorización:** Arquitectura. **Entregables:** Migración 0001 aplicada; tests cross-tenant verdes. **Checklist:**

- ☐ RLS verificada.
 - ☐ Helper auth_business_id() funciona.
 - ☐ Seed users creados.
 - ☐ Tests cross-tenant.
-

Fase 2 — Modelo DGII/e-CF

Objetivo: Schema completo de tablas DGII con RLS.

Descripción: dgii_settings , dgii_certificates , ecf_sequences , electronic_invoices , electronic_invoice_items , dgii_submissions , dgii_status_logs , audit_logs . Todas con business_id + RLS.

Qué se construye: Migración(es) DDL aditivas; función reserve_next_encf atómica.

Archivos: supabase/migrations/0002_phase2_inventory.sql , 0003_dgii_pos.sql . **Tablas:** todas las DGII.

Variables: ninguna nueva. **Agentes:** Supabase/RLS, Compliance, Backend e-CF. **Skills:** Postgres avanzado, RLS, JSONB. **Dependencias:** Fase 1. **Riesgos:** FK incorrecta, RLS faltante, índice missing. **Validaciones:** SELECT * FROM pg_policies WHERE schemaname='public' revisa cada tabla DGII. **Pruebas:** Smoke INSERT/SELECT con seed users. **Criterios de salida:** Todas las tablas tienen RLS; función reserve_next_encf testeada (atómica bajo concurrencia). **Qué NO hacer:** Subir cert real; cargar sequences reales. **Autorización:** Supabase agent.

Entregables: Migraciones aplicadas + tests. **Checklist:**

- ☐ Migración aplicada.
 - ☐ RLS verificada en cada tabla.
 - ☐ Función reserve_next_encf testeada.
 - ☐ Smoke cross-tenant.
-

Fase 3 — UI mock/demo

Objetivo: Pantallas DGII funcionando con datos fake.

Descripción: /dgii overview, /dgii/habilitacion (wizard inicial), /dgii/certificado (form mock), /dgii/configuracion, /dgii/secuencias, /dgii/certificacion (pre-cert mock con 4 tipos).

Qué se construye: Componentes React, mocks lib/mock-data/* , stores localStorage.

Archivos: apps/web/src/app/(app)/dgii/** , apps/web/src/features/dgii/*-store.ts . **Tablas:** ninguna real; localStorage. **Variables:** DATA_SOURCE=mock . **Agentes:** Frontend/UX, Producto. **Skills:** React RSC + client, Tailwind, UX writing. **Dependencias:** Fase 0 (PRD). **Riesgos:** UX confusa, copy técnico, falta de mensajes de error. **Validaciones:** QA manual sigue mockups. **Pruebas:** Test de render + interacción básica. **Criterios de**

salida: Todas las pantallas DGII cargan; flujo wizard 100% navegable. **Qué NO hacer:** Conectar Supabase real (Fase 7); cert real (Fase 9). **Autorización:** Producto + Frontend. **Entregables:** UI completa modo mock.

Checklist:

- ☐ Wizard navegable.
 - ☐ Mensajes en español RD.
 - ☐ Mobile responsive.
 - ☐ Sin errores en consola.
-

Fase 4 — Builder XML e-CF

Objetivo: `buildEcFXML(input)` que produce XML XSD-compliant para tipos 31/32/33/34.

Descripción: Service puro TypeScript que recibe `EcfBuilderInput` (RNC, razón social, items, totales, etc.) y devuelve XML válido contra XSD oficial.

Qué se construye: `apps/web/src/server/services/dgii/builder.ts` + tests exhaustivos.

Archivos: `builder.ts`, `builder.test.ts`, `types.ts`. **Tablas:** ninguna. **Variables:** ninguna. **Agentes:** Backend e-CF, XML/XSD, Compliance. **Skills:** XML/XSD, `xmlbuilder2`, TypeScript. **Dependencias:** XSDs oficiales en `docs/dgii/xsd/`. **Riesgos:** Orden de elementos incorrecto, encoding mal, regex de validación off.

Validaciones: Tests por tipo con XSD oficial. **Pruebas:** Vitest > 100 cases (input válido / inválido / edge cases).

Criterios de salida: Tests verdes para 4 tipos; XSD oficial valida output. **Qué NO hacer:** Firmar (Fase 6); enviar a DGII (Fase 14). **Autorización:** Backend. **Entregables:** `builder.ts` + `builder.test.ts`. **Checklist:**

- ☐ 4 tipos soportados.
 - ☐ XSD oficial valida.
 - ☐ Tests > 100.
 - ☐ Errores tipados (`EcfBuilderInvalidInput`, `EcfBuilderUnsupported`).
-

Fase 5 — XSD validation

Objetivo: `validateEcFXML({xml, xsd})` que valida un XML contra XSD oficial vía `xmllint-wasm`.

Descripción: Wrapper alrededor de `xmllint-wasm` que normaliza XSD oficial (tipo del XSD 31 conocido) y devuelve `{valid, errors[]}`.

Qué se construye: `validator.ts` + `validator.test.ts`.

Archivos: `validator.ts`, `validator.test.ts`. **Variables:** ninguna. **Agentes:** XML/XSD, Backend. **Skills:** XSD, `xmllint`, WebAssembly. **Dependencias:** `xmllint-wasm` dep + Fase 4. **Riesgos:** XSD desactualizado, parche en disco (mal — debe ser en memoria). **Validaciones:** XML firmado del builder pasa; XML mal-formado falla con error específico. **Pruebas:** Tests por tipo + cases de error. **Criterios de salida:** `validateEcFXML` estable; XSDs no modificados en disco. **Qué NO hacer:** Validar XMLs externos sin sanitizar. **Autorización:** Backend. **Entregables:** `validator.ts`. **Checklist:**

- ☐ `xmllint-wasm` instalado.

- ☐ Tests por tipo.
 - ☐ patchOfficialDgiiXsd para typo XSD 31.
-

Fase 6 — Firma digital demo

Objetivo: `signEcfXml({xml, certPem, privateKeyPem})` que firma con XMLDSig enveloped.

Descripción: Wrapper de xml-crypto con configuración DGII: enveloped, c14n, RSA-SHA256, KeyInfo X509, Reference URI vacío con isEmptyUri.

Qué se construye: `signer.ts` + `signer.test.ts` + `verifyEcfSignature` .

Archivos: `signer.ts` , `signer.test.ts` . **Agentes:** Firma Digital, Backend, Seguridad. **Skills:** XMLDSig, c14n, RSA, X.509. **Dependencias:** xml-crypto, @xmldom/xmldom. **Riesgos:** Id agregado al ECF (rechaza XSD), c14n incorrecta. **Validaciones:** XSD oficial valida XML firmado; verifyEcfSignature retorna true. **Pruebas:** Roundtrip sign + verify; XSD post-firma. **Criterios de salida:** XSD oficial acepta XML firmado. **Qué NO hacer:** Usar cert real (Fase 9); enviar a DGII (Fase 14). **Autorización:** Backend + Seguridad. **Entregables:** signer.ts. **Checklist:**

- ☐ isEmptyUri:true.
 - ☐ XSD post-firma OK.
 - ☐ verifyEcfSignature implementado.
-

Fase 7 — Supabase real + RLS

Objetivo: Conectar el SaaS a Supabase Preview con RLS.

Descripción: `DATA_SOURCE=supabase` activado en Preview env vars. Auth real con seed user. Helper `auth_business_id()` lee JWT claims.

Qué se construye: Migraciones aplicadas + scripts bootstrap seed user + env vars Preview.

Archivos: `scripts/bootstrap-preview-supabase-user.mjs` . **Variables:** `NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` , `DATA_SOURCE=supabase` (Preview only). **Agentes:** Supabase, Seguridad, DevOps. **Skills:** Supabase Auth, JWT claims, RLS, Vercel env. **Dependencias:** Fase 2. **Riesgos:** JWT claims faltantes; RLS rota; service_role expuesto. **Validaciones:** `/api/health` responde `data_source=supabase`; smoke 11/11 rutas verdes. **Pruebas:** Test cross-tenant con seed users distintos. **Criterios de salida:** Auth real funciona; RLS aplica; 11/11 rutas en preview verdes. **Qué NO hacer:** Activar `DATA_SOURCE=supabase` en Production (sigue mock). **Autorización:** DevOps + Seguridad. **Entregables:** Preview operativo con Supabase real.

Checklist:

- ☐ Seed user creado.
 - ☐ `/api/health` verde.
 - ☐ 11/11 rutas verdes.
 - ☐ Production env intacto.
-

Fase 8 — Wizard habilitación SaaS

Objetivo: /dgii/habilitacion con 10 pasos, panel global, banner Modo SaaS.

Descripción: Wizard completo con `enablement-store` (localStorage), evaluator con 8 estados globales, EnablementStepCard, paso "Autorización del representante e-CF" con form de evidencia rica + declaración.

Qué se construye: Múltiples componentes y stores.

Archivos: `app/(app)/dgii/habilitacion/page.tsx` , `features/dgii/enablement-store.ts` , `features/dgii/enablement-evaluator.ts` , `components/dgii/enablement-step-card.tsx` , `components/dgii/representante-evidence-form.tsx` , `components/dgii/testecf-preflight-runner.tsx` . **Tablas:** localStorage (futuro: tabla Supabase). **Agentes:** Frontend, Producto, Soporte. **Skills:** React avanzado, UX, accesibilidad. **Dependencias:** Fase 3, 7. **Riesgos:** UX confusa al cliente; evaluator con gates flojos. **Validaciones:** QA manual completo del wizard. **Pruebas:** Vitest del store + evaluator + form de evidencia. **Criterios de salida:** Wizard navegable; evaluator con anti-bypass; tests verdes. **Qué NO hacer:** Tocar DGII real (Fase 12+); usar cert real (Fase 9). **Autorización:** Producto + Frontend. **Entregables:** Wizard completo. **Checklist:**

- ☐ 10 pasos visibles.
 - ☐ Form evidencia funcional.
 - ☐ Declaración formal.
 - ☐ Anti-bypass tested.
-

Fase 9 — Certificado real en Preview

Objetivo: Subir un `.p12` real cifrado a Supabase Storage en Preview.

Descripción: Endpoint `/api/dgii/certificate/upload` que recibe FormData, valida MIME, parsea con node-forge, cifra blob+password con AES-256-GCM, persiste en Storage + `dgii_certificates`.

Qué se construye: Endpoint, service `certificate-storage.ts` , helper `cert-cipher.ts` .

Archivos: `app/api/dgii/certificate/upload/route.ts` , `server/services/certificate-storage.ts` , `server/crypto/cert-cipher.ts` . **Tablas:** `dgii_certificates` , `audit_logs` . **Variables:** `DGII_CERT_ENCRYPTION_KEY` (32 bytes random base64). **Agentes:** Seguridad/Certificados, Backend, Compliance. **Skills:** PKCS#12, AES-GCM, Supabase Storage. **Dependencias:** Fase 7. **Riesgos:** Leak de password, blob no cifrado, key expuesta. **Validaciones:** `dgii_certificates` fila creada con `pkcs12_encrypted_blob` cifrado; audit log presente. **Pruebas:** Smoke con cert dummy. **Criterios de salida:** Cert real subido y persistido; sin leak en logs. **Qué NO hacer:** Subir a Production env (Preview only); enviar a DGII (Fase 14). **Autorización:** Seguridad + cliente del SaaS. **Entregables:** Endpoint + service. **Checklist:**

- ☐ AES-256-GCM funciona.
 - ☐ Sin leak en logs.
 - ☐ Cert persistido.
 - ☐ Audit log.
-

Fase 10 — Validación local certificado

Objetivo: `runLocalCertTest` que valida cert real con XSD oficial + firma + verificación.

Descripción: Service async que: descifra cert, parsea metadata, build XML demo, firma RSA-SHA256, verifica, valida estructura, XSD oficial, QR demo. 8 steps verdes.

Qué se construye: `local-cert-test.ts` + tests + endpoint POST + UI `certificado-real.tsx`.

Archivos: `server/services/local-cert-test.ts`, `app/api/dgii/certificate/test-local/route.ts`, `features/dgii/components/certificado-real.tsx`.

Agentes: Backend, Seguridad, QA Auto. **Skills:** PKCS#12, XMLDSig, XSD. **Dependencias:** Fase 4, 5, 6, 9. **Riesgos:** Leak en evidencia, XSD path no encontrado en Vercel.

Validaciones: 8 steps verdes con cert real; sin password en evidence. **Pruebas:** Test con cert generado dinámicamente. **Criterios de salida:** Prueba local funciona; XSD step pasa con cert real. **Qué NO hacer:** Llamar DGII (Fase 14). **Autorización:** Backend. **Entregables:** Service + endpoint + UI. **Checklist:**

- ☐ 8 steps verdes.
 - ☐ XSD valida.
 - ☐ Sin leak.
 - ☐ `outputFileTracingIncludes` para XSDs.
-

Fase 11 — Pre-Fase G checklist cliente/contador

Objetivo: Paso 8 wizard con 9 ítems de evidencia + declaración formal del responsable.

Descripción: UI `RepresentanteEvidenceForm` permite registrar por ítem: estado tri-state, responsable, fecha, ref documental, nota. Declaración bloquea `ready_for_testecf`.

Qué se construye: `representante-evidence-form.tsx`, extensión de `enablement-store` con `ChecklistItemEvidence` y `declarationAccepted`, evaluator estricto.

Archivos: `components/dgii/representante-evidence-form.tsx`, `features/dgii/enablement-store.ts`, `features/dgii/enablement-evaluator.ts`. **Agentes:** Frontend, Producto, Contable, Soporte. **Skills:** UX para checklists, React forms, persistence. **Dependencias:** Fase 8. **Riesgos:** Bypass del gate; UX confusa.

Validaciones: Test anti-bypass (Select=completed sin evidence → in_progress). **Pruebas:** Vitest store + evaluator + UI. **Criterios de salida:** Anti-bypass funciona; declaración requerida. **Qué NO hacer:** Avanzar Fase G sin esto. **Autorización:** Producto. **Entregables:** Paso 8 completo. **Checklist:**

- ☐ 9 ítems con evidencia.
 - ☐ Declaración formal.
 - ☐ Anti-bypass.
 - ☐ QA manual 14/14.
-

Fase 12 — Dry-run Fase G

Objetivo: `prepareTestecfSubmission` que arma payload completo sin tocar DGII.

Descripción: Service async puro que valida ambiente, build XML, valida XSD, firma, computa URLs testecf, devuelve evidencia + razones de bloqueo. `executeTestecfSubmission` stub que siempre tira `TestecfSendDisabled` . Endpoint POST + UI.

Qué se construye: `testecf-client.ts` , `testecf-preflight.ts` , endpoint, UI `testecf-preflight-runner.tsx` , tests con `vi.spyOn(fetch)` que tira.

Archivos: `server/services/dgii/testecf-client.ts` , `testecf-preflight.ts` , `app/api/dgii/invoices/testecf-send/route.ts` , `components/dgii/testecf-preflight-runner.tsx` . **Variables:** `DGII_BASE_URL_TESTECF` (default hardcoded), `DGII_TESTECF_SEND_ENABLED=false` . **Agentes:** Backend, Integración testecf, QA Auto, Seguridad. **Skills:** HTTP design, killswitches, testing avanzado. **Dependencias:** Fase 10, 11. **Riesgos:** Fetch accidental, leak en preflight evidence. **Validaciones:** Test `fetchSpy.not.toHaveBeenCalled`; refuse ambiente `!== testecf`; refuse override `/ecf/`. **Pruebas:** Vitest 11+ tests del cliente + tests guards. **Criterios de salida:** Dry-run funciona end-to-end; zero fetch en suite; killswitches operan. **Qué NO hacer:** Implementar fetch real (Fase 14). **Autorización:** Backend + Seguridad. **Entregables:** Cliente dry-run + UI + tests. **Checklist:**

- ☐ `prepareTestecfSubmission` OK.
 - ☐ `executeTestecfSubmission` stub tira siempre.
 - ☐ killswitch flag funciona.
 - ☐ Guard global fetch.
-

Fase 13 — Postulación + secuencias e-NCF

Objetivo: Cliente / contador completan trámite DGII y cargan rangos.

Descripción: El cliente entra a SIRTSS DGII, postula al ambiente testecf con sus URLs internas, recibe rangos e-NCF asignados por DGII. El sistema solo gestiona el alta del rango en `ecf_sequences` .

Qué se construye: UI `/dgii/secuencias` con form para alta del rango; validaciones; función SQL `reserve_next_encf` ya implementada en Fase 2.

Archivos: `app/(app)/dgii/secuencias/page.tsx` . **Tablas:** `ecf_sequences` . **Agentes:** Contable, Cliente, Frontend. **Skills:** SIRTSS DGII, validación de rangos. **Dependencias:** Postulación aprobada DGII (externo). **Riesgos:** Cliente carga rango incorrecto; rango expira. **Validaciones:** Validación `range_start < range_end`; alarma 90%. **Pruebas:** Smoke de reserva atómica. **Criterios de salida:** Rango cargado y `status='active'` ; postulación documentada. **Qué NO hacer:** Continuar Fase 14 sin esto. **Autorización:** Contador + dueño. **Entregables:** `ecf_sequences` poblada. **Checklist:**

- ☐ Postulación DGII aprobada.
 - ☐ Rango cargado por tipo.
 - ☐ `expires_at` registrado.
 - ☐ Alarmas configuradas.
-

Fase 14 — Fase G testecf real

Objetivo: Implementar HTTP real del cliente DGII testecf y enviar primer e-CF.

Descripción: `executeTestecfSubmission` deja de ser stub; implementa Semilla → ValidarSemilla → token → POST multipart recepción. Persiste `dgii_submissions`. Confirma `TrackId`.

Qué se construye: Cuerpo HTTP del client, token cache, persistencia, error handling, retry policy.

Archivos: `testecf-client.ts` (extender), `server/services/dgii/testecf-auth.ts`, route `/api/dgii/invoices/[id]/send`. **Variables:** `DGII_TESTECF_SEND_ENABLED=true` (Preview only, autorizado).

Agentes: Integración testecf, Backend, Seguridad, Compliance. **Skills:** HTTP avanzado, multipart, error handling, OAuth-like. **Dependencias:** Fase 12 + Fase 13 + autorización per-tipo. **Riesgos:** Endpoint prod accidental, leak token, rate limit DGII. **Validaciones:** Tests con fetch mockeado para cada error 4xx/5xx; primer envío real con tipo 31. **Pruebas:** Vitest + smoke testecf real. **Criterios de salida:** `TrackId` guardado para tipo 31; cero leak de tokens; persistencia en `dgii_submissions`. **Qué NO hacer:** Fase H polling sin autorización separada; certecf. **Autorización:** EXPLÍCITA per-tipo del dueño + contador. **Entregables:** Cliente real + primer envío exitoso. **Checklist:**

- ☐ Auth flow Semilla → ValidarSemilla.
 - ☐ POST multipart implementado.
 - ☐ `TrackId` recibido.
 - ☐ `dgii_submissions` poblada.
 - ☐ Audit log.
-

Fase 15 — Fase H TrackId/status

Objetivo: Polling de status de DGII para invoices con `TrackId`.

Descripción: Edge Function cron consulta `/consultatrackid/api/TrackIds/Estado` cada N min para `electronic_invoices.status IN ('submitted','in_process')`. Actualiza status + INSERT `dgii_status_logs`.

Qué se construye: Edge Function cron, status client, mapeo DGII → status interno.

Archivos: `app/api/dgii/invoices/[id]/status/route.ts` + Edge Function de cron. **Tablas:** `dgii_status_logs`, update `electronic_invoices`. **Variables:** Vercel Cron config. **Agentes:** `TrackId/Status`, Observabilidad, DevOps. **Skills:** Edge Functions, cron, rate limit. **Dependencias:** Fase 14. **Riesgos:** Polling abusivo (DGII 429), leak token en cron logs. **Validaciones:** Rate limit testeado; logs sin tokens. **Pruebas:** Mock DGII responses (Aceptado/Rechazado/EnProceso); test rate limit. **Criterios de salida:** Status sincronizado < 10 min de envío; cron operativo. **Qué NO hacer:** Polling masivo (max 60/min por business). **Autorización:** Backend + DevOps. **Entregables:** Cron + status client. **Checklist:**

- ☐ Cron activo.
 - ☐ Rate limit aplicado.
 - ☐ Mapeo DGII → status correcto.
 - ☐ `dgii_status_logs` poblada.
-

Fase 16 — Certificación DGII

Objetivo: DGII certifica formalmente al contribuyente para emitir en ambiente certecf.

Descripción: Cliente / contador envía X comprobantes de cada tipo en testecf, DGII evalúa, emite certificación formal con acta. El SaaS cambia `ambiente='certecf'` para ese business.

Qué se construye: UI para cambiar ambiente con doble confirmación + audit; documentación de la transición.

Archivos: `app/(app)/dgii/configuracion/page.tsx` (extender). **Tablas:** update `dgii_settings.ambiente`.

Agentes: Contable, Compliance, Orquestador. **Skills:** SIRTSS DGII, normativa, gestión documental.

Dependencias: Fase 14, 15 estables con tasa aceptación > 95%. **Riesgos:** Transición prematura; acta no archivada; rangos certecf distintos. **Validaciones:** Acta DGII subida al sistema; audit log de la transición.

Pruebas: Smoke en certecf (con autorización explícita). **Criterios de salida:** Acta DGII archivada; ambiente=certecf; certificación en curso. **Qué NO hacer:** Saltar a producción (Fase 17) sin acta DGII.

Autorización: EXPLÍCITA del dueño + contador + DGII (acta). **Entregables:** Acta DGII + UI ambiente switcher.

Checklist:

- ☐ Acta archivada.
 - ☐ Ambiente switchado.
 - ☐ Audit log.
 - ☐ Rangos certecf cargados.
-

Fase 17 — Producción fiscal

Objetivo: Activar `ambiente='ecf'` + `dgii_enabled_real_send=true` para el cliente.

Descripción: Cambio final a producción fiscal después de certificación DGII formal. Comprobantes emitidos son fiscalmente válidos. Backups, alarmas, retención y plan de rollback activos.

Qué se construye: UI de activación con triple confirmación (dueño + admin + contador); checklist § 26 del PRD verificado.

Archivos: UI; runbook producción. **Tablas:** update `dgii_settings`. **Variables:** `DGII_BASE_URL_ECF` configurada en Vercel Production (per cliente). **Agentes:** Orquestador, Seguridad Producción, DevOps, Compliance. **Skills:** Cambio de producción, monitoreo. **Dependencias:** Fase 16 + checklist go-live verde (15 items PRD § 31).

Riesgos: Activación prematura; emisión fiscal incorrecta; rollback complicado. **Validaciones:** Checklist go-live 15/15 verde; backup verificado restorable; alarmas testeadas. **Pruebas:** Smoke fiscal con 1 comprobante real + reversal documentado. **Criterios de salida:** Producción fiscal activa para el cliente; primer e-CF fiscal emitido y aceptado. **Qué NO hacer:** Activar para múltiples clientes a la vez (uno por uno). **Autorización:** EXPLÍCITA triple (dueño SaaS + admin cliente + contador) + verificación compliance. **Entregables:** Cliente en producción fiscal.

Checklist:

- ☐ Checklist go-live 15/15.
- ☐ Backup restorable.
- ☐ Alarmas activas.
- ☐ Primer e-CF fiscal aceptado.

Fase 18 — Operación, soporte y monitoreo

Objetivo: Operación continua estable: cliente emitiendo, soporte resolviendo, monitoreo activo.

Descripción: Monitoreo de KPIs (tasa aceptación, latencia, cert vencimiento), alarmas, runbooks de incidentes, soporte L1/L2 capacitado.

Qué se construye: Dashboards, alarmas, runbooks de soporte, FAQs.

Archivos: docs/dgii/runbook-soporte.md , docs/dgii/runbook-incidentes.md , FAQ. **Agentes:** Soporte, Observabilidad, Incidentes, Capacitación. **Skills:** Observability, comunicación, gestión de crisis. **Dependencias:** Fase 17. **Riesgos:** Soporte no capacitado, alarmas ruidosas, MTTR alto. **Validaciones:** SLA cumplidos; NPS clientes > 50. **Pruebas:** Simulacro DGII outage. **Criterios de salida:** Operación estable 30 días sin incidentes P0; SLA cumplidos. **Qué NO hacer:** Avanzar a Fase 19 sin operación estable. **Autorización:** Continuo. **Entregables:** Dashboards + runbooks + FAQ. **Checklist:**

- ☐ Dashboards activos.
- ☐ Alarmas testeadas.
- ☐ Runbooks publicados.
- ☐ Soporte capacitado.

Fase 19 — Reutilización en otros SaaS

Objetivo: Empaquetar el módulo para integrarlo en otros SaaS RD.

Descripción: Refactor del módulo a paquete portable (services + migraciones + UI templates + tests). Doc de adaptación. Caso piloto en SaaS distinto.

Qué se construye: Paquete npm interno o estructura de copy-paste documentada.

Archivos: packages/saas-dgii/ (si monorepo) o docs/dgii/portable-package.md . **Agentes:** Migración entre SaaS, Arquitectura, Documentación. **Skills:** Refactor avanzado, design system agnostic. **Dependencias:** Fase 18 estable. **Riesgos:** Acoplamiento residual con DermaLand; tests que no portan. **Validaciones:** Caso piloto en SaaS B funciona. **Pruebas:** Tests del paquete sin deps de DermaLand específicas. **Criterios de salida:** Segundo SaaS usando el módulo. **Qué NO hacer:** Romper DermaLand al refactor. **Autorización:** Orquestador. **Entregables:** Paquete + segundo SaaS piloto. **Checklist:**

- ☐ Paquete portable.
- ☐ Tests verdes en piloto.
- ☐ Docs de adaptación.

Fase 20 — Escalamiento multi-cliente

Objetivo: Operar el módulo con N clientes simultáneos sin degradación.

Descripción: Optimización de queries por business, índices, caching, edge functions paralelas, observabilidad multi-tenant.

Qué se construye: Optimizaciones de performance, caching, dashboards multi-tenant.

Archivos: Refactor de queries; índices DB; caching. **Agentes:** Arquitectura, Observabilidad, DevOps. **Skills:** Performance, caching, observability avanzada. **Dependencias:** Fase 18 + Fase 19. **Riesgos:** Cuello de botella en cron; tenant ruidoso afecta a otros. **Validaciones:** Carga simulada con 100 businesses concurrentes. **Pruebas:** Load testing. **Criterios de salida:** SLA mantenido a 100+ businesses. **Qué NO hacer:** Multi-region sin compliance review. **Autorización:** Arquitectura. **Entregables:** Sistema escalado. **Checklist:**

- ☐ Performance OK a 100 tenants.
 - ☐ Cron sin colisiones.
 - ☐ Tenant isolation bajo carga.
-

3. Reglas de transición entre fases

- **No se salta una fase.** Si Fase N-1 no está verde, no se empieza N.
 - **Cada fase requiere autorización explícita.** Documentada en commit o PROJECT_MEMORY.
 - **Killswitches se mantienen entre fases.** No se desactiva un killswitch porque "ya estamos en la fase siguiente".
 - **Producción Vercel intacta hasta Fase 17.** Cero env vars de Production hasta certificación DGII formal.
 - **Cada transición se audita.** audit_log con `action='dgi_phase_transition'`.
 - **Rollback documentado por fase.** Antes de avanzar, se valida que se puede rollback.
-

4. Adaptación a otro SaaS

Otro SaaS RD empezando desde cero: seguir las 20 fases tal cual.

Otro SaaS RD con sistema multi-tenant existente: saltar Fase 1; auditar en Fase 0 que tiene RLS + business_id; continuar desde Fase 2.

SaaS para otro país (México CFDI, Chile DTE, Perú CPE): la estructura aplica; los XSDs, endpoints y normativa cambian. Reusar:

- Patrón multi-tenant + RLS.
- Estructura de fases.
- Killswitches + dry-run.
- Cifrado de cert.
- UX del wizard.

Reescribir:

- Builder XML específico del país.

- XSDs del país.
- Endpoints HTTP del país.
- Roles fiscales (Usuario Admin e-CF es específico de DGII RD).

Fin del Roadmap. Ver `checklist-implementacion-saas-dgii.md` para tareas concretas.

Capítulo 5 — Checklist de Implementación

Checklist de Implementación — SaaS DGII e-CF para RD

Versión: 1.0 · **Fecha:** 2026-05-21 30 checklists prácticos con responsable + evidencia + criterio de salida + riesgo si no se cumple.

Introducción

Este documento es la guía operativa para implementar el módulo SaaS DGII e-CF en otro proyecto. Cada checklist contiene tareas con:

- **Responsable:** agente o rol que ejecuta (ver `agentes-y-skills-saas-dgii.md`).
- **Evidencia requerida:** qué debe quedar como prueba (archivo, output, screenshot, commit).
- **Criterio de salida:** condición binaria que indica que está hecho.
- **Riesgo si no se cumple:** impacto en compliance, seguridad u operación.

Símbolos: `[ ]` pendiente · `[x]` hecho · `[-]` no aplica al proyecto.

Checklist 1 — Inicial del proyecto

- ☐ **Kickoff con stakeholders (dueño, contador, técnico)**
 - Responsable: Orquestador
 - Evidencia: Acta de reunión + lista de stakeholders
 - Criterio: Todos los stakeholders identificados y comprometidos
 - Riesgo: Avanzar sin alineación → re-trabajo
- ☐ **Alcance documentado en PRD**
 - Responsable: Producto
 - Evidencia: `docs/dgii/prd-saas-facturacion-electronica-dgii.md` adaptado
 - Criterio: PRD firmado por dueño SaaS
 - Riesgo: Scope creep
- ☐ **Autorizaciones iniciales firmadas**
 - Responsable: Orquestador
 - Evidencia: Email / firma del dueño autorizando construir el módulo
 - Criterio: Autorización archivada

- Riesgo: Construir sin mandato
 - ☐ **Plan Maestro adaptado al proyecto**
 - Responsable: Orquestador
 - Evidencia: `docs/dgii/plan-maestro-saas-dgii.md` con secciones específicas
 - Criterio: Plan revisado por arquitectura
 - Riesgo: Decisiones inconsistentes
 - ☐ **Equipo asignado por agente**
 - Responsable: Orquestador
 - Evidencia: Matriz nombre → agente (`agentes-y-skills-saas-dgii.md`)
 - Criterio: Todos los 25 agentes con asignación (humano o IA)
 - Riesgo: Tareas sin owner
-

Checklist 2 — Arquitectura

- ☐ **ADR de stack (Next.js + Supabase + Vercel + node-forge + xml-crypto + xmllint-wasm)**
 - Responsable: Arquitectura
 - Evidencia: `docs/architecture/ADR-001-stack.md`
 - Criterio: ADR firmado
 - Riesgo: Cambios de stack a medio camino
 - ☐ **Diagrama multi-tenant documentado**
 - Responsable: Arquitectura
 - Evidencia: Diagrama ASCII / Mermaid en plan maestro
 - Criterio: Diagrama explica capas + flujos
 - Riesgo: Acoplamiento entre tenants
 - ☐ **Decisiones técnicas (XMLDSig enveloped, isEmptyUri, multipart) documentadas**
 - Responsable: Arquitectura + Backend
 - Evidencia: ADRs específicos
 - Criterio: Cada decisión con justificación
 - Riesgo: Re-implementación con decisión equivocada
 - ☐ **Estructura de carpetas definida**
 - Responsable: Arquitectura
 - Evidencia: `apps/web/src/server/services/dgii/` , `app/(app)/dgii/` , `app/api/dgii/` , `docs/dgii/`
 - Criterio: Convención clara
 - Riesgo: Inconsistencia entre PRs
-

Checklist 3 — Supabase

- ☐ **Proyecto Supabase creado**
 - Responsable: DevOps
 - Evidencia: `SUPABASE_PROJECT_REF` documentado
 - Criterio: Acceso Studio + API funcional
 - Riesgo: Bloqueo de Fase 1
 - ☐ **Migraciones 0001-0007 escritas y aplicadas**
 - Responsable: Supabase/RLS
 - Evidencia: `supabase/migrations/000[1-7]*.sql` + `pg_policies` query
 - Criterio: Migraciones idempotentes; tablas con RLS
 - Riesgo: Schema inconsistente entre entornos
 - ☐ **Backups diarios + retención 10 años configurados**
 - Responsable: DevOps + Compliance
 - Evidencia: Configuración Supabase Backups + retención mensual cifrada
 - Criterio: Restore probado en stage
 - Riesgo: Pérdida de evidencias fiscales
 - ☐ **Región Supabase elegida (us-east-1 cercana a DGII)**
 - Responsable: Arquitectura
 - Evidencia: Configuración region
 - Criterio: Latencia < 200ms DGII
 - Riesgo: Latencia alta en flow submit
-

Checklist 4 — RLS

- ☐ **Todas las tablas DGII tienen `business_id` NOT NULL**
 - Responsable: Supabase/RLS
 - Evidencia: `\d <tabla>` + query a `information_schema`
 - Criterio: Sin excepciones (excepto `businesses`)
 - Riesgo: Cross-tenant leak
- ☐ **Policy `_all` con `USING + WITH CHECK = business_id = auth_business_id()` en cada tabla**
 - Responsable: Supabase/RLS
 - Evidencia: `SELECT * FROM pg_policies` por tabla
 - Criterio: Policy presente en cada tabla
 - Riesgo: Sin enforcement RLS
- ☐ **`audit_logs` tiene INSERT policy con `user_id = auth.uid()`**

- Responsable: Auditoría
 - Evidencia: Migración 0007 aplicada
 - Criterio: Suplantación bloqueada
 - Riesgo: Auditoría falseable
 - ☐ **Smoke test cross-tenant en CI**
 - Responsable: QA Auto
 - Evidencia: Test vitest + seeder 2 businesses
 - Criterio: User A NO puede SELECT/INSERT en business B
 - Riesgo: Regresión silenciosa
-

Checklist 5 — Vercel

- ☐ **Proyecto Vercel link a repo**
 - Responsable: DevOps
 - Evidencia: `.vercel/project.json` (gitignored)
 - Criterio: Preview deploy automático por PR
 - Riesgo: Deploys manuales propensos a error
- ☐ **Root Directory configurado en `apps/web` (monorepo)**
 - Responsable: DevOps
 - Evidencia: Vercel project settings
 - Criterio: Build funciona desde `apps/web`
 - Riesgo: Build falla en monorepo
- ☐ **Production env vars VACÍAS al inicio**
 - Responsable: DevOps + Compliance
 - Evidencia: `vercel env ls production` retorna lista vacía o solo no-sensibles
 - Criterio: `DATA_SOURCE` ausente → defaultea a mock
 - Riesgo: Activación accidental de prod
- ☐ **Preview env vars seteadas (Supabase URL, anon key, DGII_CERT_ENCRYPTION_KEY)**
 - Responsable: DevOps
 - Evidencia: `vercel env ls preview` confirma vars sin imprimir valores
 - Criterio: Preview deploy funcional
 - Riesgo: Preview no opera
- ☐ **`outputFileTracingIncludes` para XSDs configurado**
 - Responsable: Backend + DevOps
 - Evidencia: `next.config.ts` con `outputFileTracingIncludes`
 - Criterio: XSDs disponibles en deploy

- Riesgo: XSD validation falla en producción
-

Checklist 6 — Variables de entorno

- ☐ `NEXT_PUBLIC_SUPABASE_URL` **(Preview) configurada**
 - Responsable: DevOps
 - Evidencia: `vercel env ls preview`
 - Criterio: URL válida
 - Riesgo: Sin conexión Supabase
 - ☐ `NEXT_PUBLIC_SUPABASE_ANON_KEY` **(Preview) configurada**
 - Responsable: DevOps
 - Evidencia: `vercel env ls preview`
 - Criterio: Key válida
 - Riesgo: Sin auth
 - ☐ `SUPABASE_SERVICE_ROLE_KEY` **(Preview, sensitive) configurada**
 - Responsable: DevOps + Seguridad
 - Evidencia: `vercel env ls --sensitive`
 - Criterio: Key marcada como sensitive
 - Riesgo: Leak service role
 - ☐ `DGII_CERT_ENCRYPTION_KEY` **(32 bytes random base64, sensitive) configurada**
 - Responsable: Seguridad + DevOps
 - Evidencia: `vercel env` + backup en Vault
 - Criterio: Key generada con `secrets.token_urlsafe(32)`
 - Riesgo: Pérdida = todos los certs irre recuperables
 - ☐ `DGII_TESTECF_SEND_ENABLED=false` **(default)**
 - Responsable: DevOps
 - Evidencia: `vercel env` / repo default
 - Criterio: Killswitch off
 - Riesgo: Envío accidental
 - ☐ `.env.example` **actualizado (sin valores reales)**
 - Responsable: Documentación
 - Evidencia: Archivo committed
 - Criterio: Lista completa de vars sin valores
 - Riesgo: Onboarding confuso
-

Checklist 7 — Multi-tenant

- ☐ **businesses como tabla raíz; otras referencian via FK**
 - Responsable: Supabase
 - Evidencia: Schema diagram
 - Criterio: Modelo consistente
 - Riesgo: Tablas huérfanas
 - ☐ **JWT contiene claim `business_id`**
 - Responsable: Supabase + Backend
 - Evidencia: Bootstrap seed user con `raw_app_meta_data.business_id`
 - Criterio: `getSession()` lee `business_id`
 - Riesgo: RLS no aplica
 - ☐ **`auth_business_id()` SQL function lee claim en orden correcto**
 - Responsable: Supabase
 - Evidencia: Migración 0006
 - Criterio: Función retorna `business_id` del JWT
 - Riesgo: Queries devuelven vacío
 - ☐ **Storage paths llevan prefijo `<business_id>/`**
 - Responsable: Backend
 - Evidencia: `certificate-storage.ts` usa pattern
 - Criterio: Sin paths sin prefijo
 - Riesgo: Cross-tenant en Storage
-

Checklist 8 — Certificados

- ☐ **Endpoint POST `/api/dgii/certificate/upload` implementado**
 - Responsable: Backend + Seguridad
 - Evidencia: `route.ts`
 - Criterio: Recibe `FormData` + valida + cifra + persiste
 - Riesgo: Subida insegura
- ☐ **Cifrado AES-256-GCM aplicado a blob `.p12` y password**
 - Responsable: Seguridad
 - Evidencia: `cert-cipher.ts` + test
 - Criterio: Blob descifrable solo con `DGII_CERT_ENCRYPTION_KEY`
 - Riesgo: Cert en claro
- ☐ **Blob cifrado persistido en Supabase Storage privado**

- Responsable: Backend
 - Evidencia: `dgii_certificates` con `pkcs12_storage_path`
 - Criterio: Storage bucket privado (sin acceso anon)
 - Riesgo: Leak vía URL pública
 - ☐ **UI muestra solo metadata pública (subject, issuer, vigencia, fingerprint short)**
 - Responsable: Frontend
 - Evidencia: `certificado-real.tsx`
 - Criterio: Sin password, sin private key, sin blob
 - Riesgo: Leak en cliente
 - ☐ **Audit log `dgii_certificate_upload` insertado**
 - Responsable: Auditoría
 - Evidencia: `audit_logs` row + RLS verde
 - Criterio: Log con metadata + sin secretos
 - Riesgo: Sin trazabilidad
-

Checklist 9 — Cifrado

- ☐ **Key de cifrado: 32 bytes random base64**
 - Responsable: Seguridad
 - Evidencia: `python -c "secrets.token_urlsafe(32)"`
 - Criterio: Length 43+ chars base64
 - Riesgo: Key débil
- ☐ **IV 12 bytes random por blob (AES-GCM)**
 - Responsable: Seguridad
 - Evidencia: `cert-cipher.ts` usa `crypto.getRandomValues`
 - Criterio: IV único por blob
 - Riesgo: Reuso → comprometer cifrado
- ☐ **Tag 16 bytes auth (AES-GCM)**
 - Responsable: Seguridad
 - Evidencia: Sealed JSON `{iv, tag, cipher}`
 - Criterio: Tag verificado al descifrar
 - Riesgo: Manipulación no detectada
- ☐ **Backup de `DGII_CERT_ENCRYPTION_KEY` en Vault físico + KMS**
 - Responsable: Seguridad
 - Evidencia: Procedimiento documentado + acta
 - Criterio: 2 ubicaciones independientes

- Riesgo: Pérdida total de certs
-

Checklist 10 — DGII

- ☐ **Norma 06-2018 y actualizaciones leídas**
 - Responsable: Compliance + Contable
 - Evidencia: Resumen + matriz de requisitos
 - Criterio: Mapeo norma → requisito sistema
 - Riesgo: Incumplimiento legal
 - ☐ **Ambientes testecf / certecf / ecf documentados con URLs**
 - Responsable: Backend + Compliance
 - Evidencia: `testecf-client.ts` con `DEFAULT_BASE_URLS`
 - Criterio: 3 URLs hardcoded; override solo via env
 - Riesgo: Endpoint incorrecto
 - ☐ **XSDs oficiales descargados a `docs/dgii/xsd/`**
 - Responsable: XML/XSD
 - Evidencia: Archivos `e-CF-N-v1.0.xsd`
 - Criterio: Tipos 31/32/33/34 mínimo
 - Riesgo: Validación incorrecta
-

Checklist 11 — RNC / Contribuyente

- ☐ **Validación de RNC (9 u 11 dígitos)**
 - Responsable: Backend
 - Evidencia: `builder.ts` con `RNC_RE`
 - Criterio: Regex `/^(?:\d{9}|\d{11})$/`
 - Riesgo: XML rechazado por XSD
- ☐ **`dgii_settings.rncEmisor` denormalizado por business**
 - Responsable: Backend
 - Evidencia: Tabla migrations 0003
 - Criterio: Acceso O(1) sin join
 - Riesgo: Performance al emitir
- ☐ **RNC del cert se valida contra RNC del business al activar cert**
 - Responsable: Compliance
 - Evidencia: `certificate-storage.ts` validación

- Criterio: Mismatch bloquea activación
 - Riesgo: Firma con cert ajeno
-

Checklist 12 — Usuario Administrador e-CF

- ☐ **Designación oficial archivada (PDF acta DGII)**
 - Responsable: Contable + Cliente
 - Evidencia: Documento en archivo del cliente
 - Criterio: Acta firmada y fechada
 - Riesgo: Sin autoridad para firmar
 - ☐ **Titular del cert coincide con designación**
 - Responsable: Compliance
 - Evidencia: Checklist § 3.7 paso 8 wizard
 - Criterio: CN del cert = nombre designado
 - Riesgo: Firma sin autorización
 - ☐ **Cédula del titular en evidencia paso 8**
 - Responsable: Cliente + Frontend
 - Evidencia: representante_attestations row
 - Criterio: Cédula registrada
 - Riesgo: Sin trazabilidad
 - ☐ **Procedimiento de revocación documentado**
 - Responsable: Compliance + Soporte
 - Evidencia: Runbook
 - Criterio: Cliente puede revocar < 24h
 - Riesgo: Cert sigue activo sin titular
-

Checklist 13 — Secuencias e-NCF

- ☐ **Rango DGII testecf cargado en `ecf_sequences`**
 - Responsable: Contable + Cliente
 - Evidencia: row con `status='active'` , ambiente='testecf'
 - Criterio: Rango oficial DGII
 - Riesgo: eNCF rechazado por DGII
- ☐ **`reserve_next_encf` función atómica (FOR UPDATE)**
 - Responsable: Supabase

- Evidencia: Migración 0003
 - Criterio: Concurrencia 100 reservas sin duplicado
 - Riesgo: eNCF duplicado
 - ☐ **Alarma 90% del rango configurada**
 - Responsable: Observabilidad
 - Evidencia: Alert rule
 - Criterio: Notifica al cliente
 - Riesgo: Agotamiento sin aviso
 - ☐ **expires_at registrado por rango**
 - Responsable: Backend
 - Evidencia: ecf_sequences schema
 - Criterio: Bloqueo automático post-vencimiento
 - Riesgo: Emisión con rango expirado
-

Checklist 14 — XML / XSD

- ☐ **buildEcfxml soporta tipos 31/32/33/34**
 - Responsable: Backend e-CF + XML/XSD
 - Evidencia: builder.test.ts cobertura por tipo
 - Criterio: Tests verdes para cada tipo
 - Riesgo: Tipo no emitible
- ☐ **validateEcfxml usa XSD oficial sin modificar archivo**
 - Responsable: XML/XSD
 - Evidencia: patchOfficialDgiiXsd en memoria
 - Criterio: XSD en disco intacto
 - Riesgo: Parche difícil de mantener
- ☐ **Orden de elementos respeta XSD**
 - Responsable: Backend
 - Evidencia: Tests que verifican orden
 - Criterio: XSD acepta
 - Riesgo: Rechazo por orden
- ☐ **Encoding UTF-8 declarado**
 - Responsable: Backend
 - Evidencia: `<?xml version="1.0" encoding="UTF-8"?>`
 - Criterio: Sin BOM
 - Riesgo: Encoding error en DGII

Checklist 15 — Firma digital

- ☐ `signEcfXml` **implementa XMLDSig enveloped**
 - Responsable: Firma Digital
 - Evidencia: `signer.ts` con `SignedXml` de `xml-crypto`
 - Criterio: Firma enveloped al final de ECF
 - Riesgo: Firma rechazada por DGII
- ☐ `SignatureMethod=rsa-sha256` , `DigestMethod=sha256` , `c14n=20010315`
 - Responsable: Firma Digital
 - Evidencia: Constants en `signer.ts`
 - Criterio: Algoritmos exactos según DGII
 - Riesgo: Verificación falla
- ☐ `Reference URI=""` **con** `isEmptyUri:true`
 - Responsable: Firma Digital
 - Evidencia: `signer.ts` `addReference`
 - Criterio: Sin Id agregado al ECF
 - Riesgo: XSD rechaza por Id
- ☐ `KeyInfo` **con** `X509Certificate` **(sin headers PEM)**
 - Responsable: Firma Digital
 - Evidencia: `getKeyInfoContent` retorna base64 puro
 - Criterio: PEM limpio sin BEGIN/END
 - Riesgo: KeyInfo mal formado
- ☐ `verifyEcfSignature` **valida la firma producida**
 - Responsable: Firma Digital + QA Auto
 - Evidencia: `signer.test.ts` `roundtrip`
 - Criterio: Sign → verify ciclo OK
 - Riesgo: Firma inconsistente

Checklist 16 — PDF / representación impresa

- ☐ `generateEcfPdf` **genera PDF con datos del e-CF**
 - Responsable: Backend + Producto
 - Evidencia: `pdf.ts` + `pdf.test.ts`
 - Criterio: PDF descargable con campos requeridos
 - Riesgo: Falta info legal

- ☐ **Campos visibles: RNC emisor, razón social, eNCF, fecha, totales, ITBIS, cliente**
 - Responsable: Producto
 - Evidencia: Test snapshot + revisión visual
 - Criterio: Todos los campos según norma DGII
 - Riesgo: Representación inválida
 - ☐ **QR DGII visible y escaneable**
 - Responsable: Backend
 - Evidencia: QR en PDF + test scan
 - Criterio: QR contiene payload oficial DGII
 - Riesgo: QR no válido
 - ☐ **Advertencia "NO FISCAL / DEMO" visible en modo mock**
 - Responsable: Producto
 - Evidencia: PDF demo con watermark
 - Criterio: Sin posibilidad de confusión con e-CF real
 - Riesgo: Emisión legal de PDF demo
-

Checklist 17 — QR / código de seguridad

- ☐ **Payload QR sigue formato oficial DGII**
 - Responsable: Backend + Compliance
 - Evidencia: qr.ts + buildDgiiConsultaUrl
 - Criterio: Validado contra norma
 - Riesgo: QR rechazado por consulta DGII
 - ☐ **Código de seguridad calculado per e-CF**
 - Responsable: Backend
 - Evidencia: security-code.ts
 - Criterio: Algoritmo según norma DGII
 - Riesgo: Código inválido
 - ☐ **Fingerprint corto del cert incluido (cuando aplica)**
 - Responsable: Backend
 - Evidencia: QR payload con `fp=<short>`
 - Criterio: Trazabilidad al cert
 - Riesgo: Sin verificación de origen
-

Checklist 18 — testecf

- ☐ **Postulación testecf aprobada DGII**
 - Responsable: Contable + Cliente
 - Evidencia: Acta SIRTSS
 - Criterio: Estado aprobado
 - Riesgo: 401 al enviar
 - ☐ **Rango e-NCF testecf asignado por DGII**
 - Responsable: Contable
 - Evidencia: Comunicación DGII + ecf_sequences row
 - Criterio: Rango activo
 - Riesgo: eNCF rechazado
 - ☐ **DGII_TESTECF_SEND_ENABLED=true (Preview only, autorizado)**
 - Responsable: DevOps + Orquestador
 - Evidencia: vercel env preview
 - Criterio: Solo cuando todo lo anterior verde
 - Riesgo: Envío sin precondiciones
 - ☐ **Dry-run preflight pasa antes de live**
 - Responsable: Backend + QA Manual
 - Evidencia: Preflight result + screenshots
 - Criterio: XSD ✓ + firma ✓ + URLs correctas
 - Riesgo: Live falla en config
-

Checklist 19 — TrackId / status

- ☐ **Edge Function cron implementada**
 - Responsable: TrackId/Status + DevOps
 - Evidencia: vercel.json cron config
 - Criterio: Cron corre cada N min
 - Riesgo: Status nunca se actualiza
- ☐ **Rate limit por business (max 60/min)**
 - Responsable: Backend
 - Evidencia: Código + test
 - Criterio: DGII 429 evitado
 - Riesgo: DGII bloquea IP
- ☐ **dgii_status_logs poblado por consulta**

- Responsable: Backend + Auditoría
 - Evidencia: row per consulta
 - Criterio: Sin huecos
 - Riesgo: Sin trazabilidad
 - ☐ **Mapeo DGII código → status interno documentado**
 - Responsable: Backend + Compliance
 - Evidencia: Tabla en runbook
 - Criterio: Todos los códigos cubiertos
 - Riesgo: Status incorrecto
-

Checklist 20 — Certificación

- ☐ **N envíos exitosos en testecf por tipo (mínimo recomendado 10/tipo)**
 - Responsable: Cliente + Contable
 - Evidencia: dgii_submissions con status=accepted
 - Criterio: Tasa aceptación > 95%
 - Riesgo: DGII rechaza certificación
 - ☐ **Solicitud de transición a certecf enviada a DGII**
 - Responsable: Contable + Cliente
 - Evidencia: Comunicación oficial
 - Criterio: Solicitud registrada
 - Riesgo: Sin trámite
 - ☐ **Acta de certificación DGII recibida y archivada**
 - Responsable: Cliente + Compliance
 - Evidencia: PDF acta
 - Criterio: Documento oficial
 - Riesgo: Operación ilegal
 - ☐ **ambiente='certecf' activado para el cliente**
 - Responsable: Backend + Orquestador
 - Evidencia: dgii_settings + audit log
 - Criterio: Cambio auditado
 - Riesgo: Continuar en testecf perdiendo trazabilidad
-

Checklist 21 — Producción fiscal

- ☐ **Acta DGII de habilitación producción archivada**

- Responsable: Cliente + Compliance
 - Evidencia: PDF acta
 - Criterio: Habilitación formal
 - Riesgo: Operación sin habilitación
 - ☐ **Rangos e-NCF de producción cargados**
 - Responsable: Contable + Cliente
 - Evidencia: ecf_sequences ambiente='ecf'
 - Criterio: Rango oficial DGII
 - Riesgo: eNCF no fiscal
 - ☐ **Checklist go-live 15/15 verde (ver PRD § 31)**
 - Responsable: Orquestador
 - Evidencia: Checklist firmado
 - Criterio: Todos los ítems verdes
 - Riesgo: Activar prematuro
 - ☐ **ambiente='ecf' + dgii_enabled_real_send=true activado**
 - Responsable: Orquestador + Triple autorización
 - Evidencia: dgii_settings + audit log
 - Criterio: Triple firma dueño SaaS + admin cliente + contador
 - Riesgo: Emisión accidental sin certificación completa
 - ☐ **Production env vars configuradas (con --sensitive)**
 - Responsable: DevOps + Seguridad
 - Evidencia: vercel env ls production
 - Criterio: Solo lo mínimo necesario
 - Riesgo: Leak en prod
-

Checklist 22 — Seguridad

- ☐ **Audit de secretos en repo (git log --all -S 'BEGIN PRIVATE')**
 - Responsable: Seguridad
 - Evidencia: Output vacío
 - Criterio: Sin material sensible en historial
 - Riesgo: Leak histórico
- ☐ **.gitignore cubre .env , .env.local , *.p12 , *.pfx , *.key , *.pem , *.crt , *.cer , *.der , *.cert , certificates/**
 - Responsable: Seguridad
 - Evidencia: .gitignore

- Criterio: Lista completa
 - Riesgo: Subir sensibles
 - ☐ **RLS verificada en cada tabla con script CI**
 - Responsable: QA Auto + Supabase
 - Evidencia: Script + reporte
 - Criterio: Sin tabla sin RLS
 - Riesgo: Leak silencioso
 - ☐ **npm audit mensual**
 - Responsable: Seguridad
 - Evidencia: Output + tickets de parches
 - Criterio: Sin vulns críticas
 - Riesgo: Vuln explotable
 - ☐ **OWASP Top 10 cubierto**
 - Responsable: Seguridad
 - Evidencia: Checklist OWASP + pentest
 - Criterio: Sin findings críticos
 - Riesgo: Breach
-

Checklist 23 — Auditoría

- ☐ **audit_logs con INSERT policy aplicada (migración 0007)**
 - Responsable: Supabase + Auditoría
 - Evidencia: Migración 0007 + smoke test
 - Criterio: INSERT funciona con `user_id = auth.uid()`
 - Riesgo: Auditoría falseable
- ☐ **Eventos auditables cubiertos (cert upload/revoke, settings update, invoice sign/send/status, attestation, ambiente switch)**
 - Responsable: Backend + Compliance
 - Evidencia: Lista en runbook + grep en código
 - Criterio: Todos los eventos críticos auditados
 - Riesgo: Sin trazabilidad
- ☐ **Retención 10 años configurada (backup mensual cifrado)**
 - Responsable: DevOps + Compliance
 - Evidencia: Política backup
 - Criterio: Recoverable hasta 10 años atrás
 - Riesgo: Incumplimiento legal

- ☐ **Exportación CSV/JSON por business + período disponible**
 - Responsable: Backend
 - Evidencia: Endpoint `/admin/auditoria/export`
 - Criterio: Output filtrado por business
 - Riesgo: Sin respuesta a auditoría externa
-

Checklist 24 — QA automatizado

- ☐ **Vitest > 400 tests verdes**
 - Responsable: QA Auto
 - Evidencia: CI output
 - Criterio: 100% pasan
 - Riesgo: Regresión silenciosa
 - ☐ **Cobertura > 90% en services (builder, signer, validator, testecf-client)**
 - Responsable: QA Auto
 - Evidencia: Coverage report
 - Criterio: 90%+
 - Riesgo: Bugs no detectados
 - ☐ **Spy global de fetch que tira si invocado**
 - Responsable: QA Auto + Seguridad
 - Evidencia: testecf-client.test.ts con vi.spyOn
 - Criterio: 0 invocaciones en CI
 - Riesgo: Test toca red
 - ☐ **Tests cross-tenant RLS**
 - Responsable: QA Auto
 - Evidencia: Test con 2 seed users
 - Criterio: A no ve B
 - Riesgo: RLS bypass
 - ☐ **Tests anti-leak (no PRIVATE KEY en evidencias)**
 - Responsable: QA Auto + Seguridad
 - Evidencia: Test regex en evidencias
 - Criterio: Sin leak
 - Riesgo: Leak inadvertido
-

Checklist 25 — QA manual

- ☐ `docs/dgii/qa-saas-pre-fase-g.md` **seguido completo**
 - Responsable: QA Manual + Cliente
 - Evidencia: Reporte 14/14
 - Criterio: Todos los criterios verdes
 - Riesgo: Bugs no detectados pre-prod
 - ☐ **Screenshots de cada paso del wizard**
 - Responsable: QA Manual
 - Evidencia: Folder de screenshots
 - Criterio: Cobertura visual
 - Riesgo: Regresión visual
 - ☐ **Validación con cliente no-técnico**
 - Responsable: Soporte + Producto
 - Evidencia: Reporte cliente
 - Criterio: Cliente completa wizard sin asistencia
 - Riesgo: UX confusa
-

Checklist 26 — Soporte cliente

- ☐ **Runbook de soporte publicado**
 - Responsable: Documentación + Soporte
 - Evidencia: `docs/dgii/runbook-soporte.md`
 - Criterio: Cubre 10 errores comunes
 - Riesgo: Soporte improvisa
- ☐ **Soporte L1 capacitado en wizard**
 - Responsable: Capacitación
 - Evidencia: Asistencia + evaluación
 - Criterio: 100% del equipo
 - Riesgo: Escalation excesiva
- ☐ **Escalation matrix definida (L1 → L2 técnico → compliance/contador)**
 - Responsable: Soporte + Orquestador
 - Evidencia: Matriz publicada
 - Criterio: Roles claros
 - Riesgo: Tickets perdidos
- ☐ **SLAs documentados (P0/P1/P2)**

- Responsable: Producto + Soporte
 - Evidencia: PRD
 - Criterio: SLAs medibles
 - Riesgo: Sin accountability
 - ☐ **FAQ pública con 20+ entradas**
 - Responsable: Soporte
 - Evidencia: FAQ.md o página
 - Criterio: Cubre dudas frecuentes
 - Riesgo: Soporte saturado
-

Checklist 27 — Contador

- ☐ **Las 4 validaciones externas firmadas:**
 - **Acta de designación Usuario Administrador e-CF archivada**
 - **Cert vigente > 60 días + sin revocación CRL/OCSP**
 - **Titular del cert autorizado para representar el RNC**
 - **RNC emisor coincide exactamente con cert + designación DGII**
 - Responsable: Contador + Cliente
 - Evidencia: 4 documentos
 - Criterio: 4/4 firmados
 - Riesgo: Operación sin autoridad fiscal
 - ☐ **Capacitación del contador en lectura de logs**
 - Responsable: Capacitación
 - Evidencia: Material + asistencia
 - Criterio: Contador puede diagnosticar rechazos DGII
 - Riesgo: Dependencia técnica permanente
 - ☐ **Exportación periódica de XMLs + respuestas DGII**
 - Responsable: Backend + Contador
 - Evidencia: Cron o manual export
 - Criterio: Backup mensual
 - Riesgo: Sin evidencia ante auditoría
-

Checklist 28 — Go-live

- ☐ **PRD § 31 (15 items) 100% verde**
 - Responsable: Orquestador

- Evidencia: Checklist firmado
 - Criterio: 15/15
 - Riesgo: Activación prematura
 - ☐ **Triple autorización (dueño SaaS + admin cliente + contador)**
 - Responsable: Orquestador
 - Evidencia: Acta firmada
 - Criterio: 3 firmas
 - Riesgo: Sin mandato completo
 - ☐ **Plan de rollback documentado y simulado**
 - Responsable: DevOps + Compliance
 - Evidencia: Runbook rollback + simulación stage
 - Criterio: Rollback < 5 min
 - Riesgo: Sin retorno seguro
 - ☐ **Comunicación a clientes finales del go-live**
 - Responsable: Producto + Soporte
 - Evidencia: Email + banner
 - Criterio: Cliente final informado
 - Riesgo: Sorpresa fiscal
 - ☐ **Oncall asignado primeros 7 días**
 - Responsable: DevOps + Soporte
 - Evidencia: Rotación
 - Criterio: 24/7 cobertura
 - Riesgo: Incidente sin respuesta
-

Checklist 29 — Post-producción

- ☐ **Monitoreo activo (dashboards + alarmas)**
 - Responsable: Observabilidad
 - Evidencia: Grafana/Vercel dashboards
 - Criterio: KPIs visibles
 - Riesgo: Operación a ciegas
- ☐ **Primer e-CF fiscal real emitido y aceptado**
 - Responsable: Cliente + Soporte
 - Evidencia: TrackId + status=accepted
 - Criterio: Confirmación DGII
 - Riesgo: Producción falla en primer envío

- ☐ **Backup verificado restorable (test mensual)**
 - Responsable: DevOps + Compliance
 - Evidencia: Reporte restore
 - Criterio: Restore < 4h
 - Riesgo: Pérdida de datos
 - ☐ **Retención fiscal funcionando (audit_logs + xml_signed_path)**
 - Responsable: Auditoría + DevOps
 - Evidencia: Política aplicada
 - Criterio: 10 años garantizados
 - Riesgo: Incumplimiento legal
 - ☐ **Métricas de operación reportadas mensual**
 - Responsable: Observabilidad + Producto
 - Evidencia: Dashboard ejecutivo
 - Criterio: Tasa aceptación, NPS, tickets
 - Riesgo: Operación sin visibilidad
-

Checklist 30 — Incidentes

- ☐ **Runbook DGII outage**
 - Responsable: Incidentes
 - Evidencia: docs/dgii/runbook-incidentes.md
 - Criterio: Pasos claros
 - Riesgo: Caos en outage
- ☐ **Procedimiento cert vencido**
 - Responsable: Soporte + Compliance
 - Evidencia: Runbook + alarmas pre-vencimiento
 - Criterio: 60/30/7 días antes
 - Riesgo: Operación bloqueada
- ☐ **Procedimiento secuencia agotada**
 - Responsable: Soporte + Contable
 - Evidencia: Runbook
 - Criterio: Cliente notificado < 24h del 90%
 - Riesgo: Bloqueo de emisión
- ☐ **Procedimiento breach de seguridad**
 - Responsable: Seguridad + Compliance + Legal

- Evidencia: IR plan
 - Criterio: Aislar + notificar < 4h
 - Riesgo: Incumplimiento legal + reputación
 - ☐ **Post-mortem template + repositorio**
 - Responsable: Incidentes + Orquestador
 - Evidencia: Template + archivo histórico
 - Criterio: Cada incidente P0/P1 con PM
 - Riesgo: Repetir mismos errores
 - ☐ **Simulacro de incidente semestral**
 - Responsable: Orquestador + Incidentes
 - Evidencia: Acta del simulacro
 - Criterio: Equipo responde según runbook
 - Riesgo: Runbook obsoleto
-

Fin del Checklist de Implementación. Usar este documento + los otros 4 del paquete para implementar el módulo SaaS DGII e-CF en cualquier proyecto.
